



A survey on real-time DAG scheduling, revisiting the Global-Partitioned Infinity War

Micaela Verucchi¹ · Ignacio Sañudo Olmedo¹ · Marko Bertogna¹

Accepted: 8 July 2023 / Published online: 14 August 2023

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

Modern cyber-physical embedded systems are characterized by a range of intricate functionalities that are subject to tight timing constraints. Unfortunately, traditional sequential task models and uniprocessors solutions are not suitable for this context: a more expressive model becomes necessary. In such cases, the Directed Acyclic Graph (DAG) is a proper model to express the complexity and the parallelism of the tasks of these kinds of systems. In recent years, several methods with different settings have been proposed to solve the schedulability problem for applications featuring DAG tasks. This paper presents a survey of the state-of-the-art of the DAG task model, focusing on scheduling tests that are more effective, easy to implement, and adopt, while providing a detailed comparison of global and partitioned scheduling solutions. Besides discussing the approaches and presenting them under a common convention, this work is the first to present a comprehensive experimental analysis of several state-of-the-art tests, thanks to the development of a C++ library that implements all the methods, which is made available at url will be inserted for camera-ready.

Keywords Real-time scheduling · DAG · Partitioned scheduling · Global scheduling

1 Introduction

Satisfying real-time requirements to guarantee predictability for automotive, avionic or aerospace applications entails finding solutions to two distinct yet complementary problems, one pertaining to software and the other to hardware. Traditionally,

✉ Micaela Verucchi
micaela.verucchi@unimore.it

Ignacio Sañudo Olmedo
ignacio.sanudoolmedo@unimore.it

Marko Bertogna
marko.bertogna@unimore.it

¹ University of Modena and Reggio Emilia, via Campi 213/B, 41125 Modena, Italy

hardware and software systems adopted in safety-related domains have been simple by design. To describe and prove the correctness of such systems, the seminal (Liu and Layland 1973) task model has been widely adopted.

The integration of computationally expensive applications in safety critical domains—primarily driven by deep learning systems—has fundamentally altered the way in which software and hardware architectures are designed and validated. In 2017 Intel released the i9-7900X processor family, including the Intel i9- 7980XE with 18 cores. The following year, AMD released the Ryzen Threadripper series of processors, among which Ryzen Threadripper 2990WX with 36 cores. In March 2020, Ampere Computing announced Ampere Altra, with the Q80-33 processor featuring 80 64-bit Arm cores. In September 2020, NVIDIA launched the GeForce RTX 3000 series of Graphic Processing Units (GPUs), including the RTX 3090, with 10496 cores. In addition to these examples, other solutions are also moving towards many-core architectures, e.g. Field Programmable Gate Array (FPGAs) or Google Tensor Processing Units (TPUs). Although most of these architectures are server oriented solutions, this trend can be found also in the embedded domain. Indeed, in the latest years, several new embedded devices have been produced and adopted in the automotive domain by established manufacturers, as Xavier or Pegasus by NVIDIA, Ultrascale+ or Versal by Xilinx, and Snapdragon by Qualcomm, but also from new ones, as Kirin from Hilisicon (owned by Huawei). The specifics of the new proposed boards are various and different, however a bottom main idea can be easily found. All of these new-generation embedded boards features a multi-core Central Processing Unit (CPU), one or more GPUs and other additional engines, such as FPGAs or Artificial Intelligence (AI) specific accelerators. Heterogeneity has become a new requirement for embedded systems, alongside the traditional goals of high performance and power management.

The applications running in such platforms are convoluted. To represent such systems the Liu and Layland task model cannot be applied, its expressiveness is limited and its adoption would lead to simplistic representations of the system, too far from reality. According to the Liu and Layland task model, a task $\tau = (C, T)$ is described only by a worst-case execution time (WCET) C and its period T , having implicit deadlines. Immediate extensions are the addition of a deadline D and the concept of sporadic tasks. Several other task models exists (Burgio et al. 2019), as the Multiframe (Mok and Chen 1996), the Generalized Multiframe (Baruah et al. 1999), the Elastic (Buttazzo et al. 1998), the Mixed Criticality (Vestal 2007), the Splitted Task (Buttazzo 2004) and the Self-Suspending Task (Chen et al. 2019). However, all these models assume single-thread execution within each task which are relatively simple and do not take into account factors such as intrinsic parallelism or precedence constraints among sub-tasks. In recent years, more complex parallel task models have been introduced, such as the Digraph Real-Time task model (Stigge et al. 2011) and the DAG task model (Baruah et al. 2012) to ensure real-time constraints on these new-generation heterogeneous systems

This paper aims to summarize the existing literature on the Directed Acyclic Graph (DAG) task model applied to real-time scheduling, focusing on the various schedulability tests proposed in recent years, from the original model proposed by Baruah et al. (2012), to the Heterogeneous Parallel Conditional DAG task proposed

by Zahaf et al. (2019, 2020). The literature on DAGs in real-time is much more extensive. The effort, hereafter reported, targets to provide a deep analysis of the task model and its extensions, cover various scheduling policies, and identify ways to determine if a task set or task is feasible on a given processing system. Hence, this survey focuses specifically on DAG-task models and their applications and variations, and does not cover other parallel task models such as fork-join, gang tasks, and digraphs. Several policies have been examined, with an emphasis on the schedulability test of DAG tasks on multi-core or heterogeneous-boards. Additionally, for partitioned scheduling, traditional partitioning strategies have been considered, and a new heuristic has been proposed that improves the overall schedulability ratio. Specifically, the purpose and contributions of this survey are:

1. to offer a clear overview of the existing literature, investigating in detail 32 different methods;
2. to compare global and partitioned scheduling solutions, both from a theoretical and an empirical point of view;
3. to understand which methods are more suitable from an industrial adoption perspective, considering the goodness of the methods both from schedulability results and execution time perspective.

To evaluate and compare the best existing methods, an implementation effort has been undertaken. This public tool is a C++ library, specifically developed for working with DAG tasks, that allows one to apply one or more of the implemented methods to a user-defined or a randomly generated taskset and can be effortlessly extended to support additional solutions. The library is publicly available.

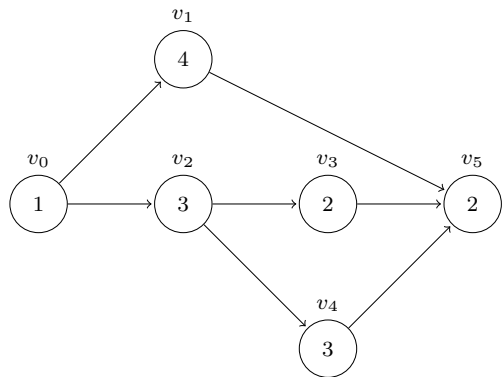
1.1 Organization

The manuscript is organized as follows: Sect. 2 introduces the DAG task model and the scheduling concepts needed to understand the specific algorithms and choices; Sect. 3 discuss global scheduling schedulability tests for DAG tasks, while the partitioned solutions can be found in Sect. 4; Sect. 5 tracts hybrid solutions, while Sect. 6 investigate extensions to the regular DAG task model. Section 7 reports all the experiments, comparing over 20 different State of The Art (SOTA) schedulability tests; their comparison is discussed in Sect. 8.

2 Background

DAGs were firstly introduced in real-time literature in the context of global scheduling by Liu and Anderson (2010). The authors investigated the Global Earliest Deadline First (G-EDF) scheduling on multiprocessors of a multi-rate task set in which tasks' precedence constraints were modeled via a DAG. The following year, the same authors further investigated the problem for distributed

Fig. 1 DAG task τ_x , with period $T_x = 16$ and deadline $D_x = 10$



systems (Liu and Anderson 2011), considering clustered scheduling and edges with assigned communication costs.

However, in both their works, the task model adopted is the sporadic one, with a given deadline and period for each task and no notion of internal parallelism.

For the sake of completeness, a recurrent task is said to be *periodic* if successive jobs of the task are required to be generated a T times units apart. A task is said *sporadic* if a lower bound, but no upper bound is specified between the generation of successive jobs (at least T times unit). Finally, an *aperiodic* task is one that occurs with no repetitions and which then does not have a specified T .

2.1 The directed acyclic graph task model

The DAG task model, has been originally presented by Baruah et al. (2012). The sporadic DAG task model is a parallel task model in which each task τ_x is represented by a tuple (G_x, D_x, T_x) , where $G_x = (V_x, E_x)$ is a vertex-weighted directed acyclic graph, and T_x and D_x are positive integers that denote the period and (relative) deadline of the task, respectively. In Fig. 1 an example of a DAG task is depicted.

Specifically, each vertex in the graph $v_i \in V_x$ corresponds to a subtask and is characterized by a worst-case execution time (WCET) C_x . The directed edges of the DAG represent the precedence constraints: if $(v, w) \in E_x$ then the job corresponding to vertex v must finish its execution before the job corresponding to vertex w can start. Different jobs may execute in parallel if they are not constrained by precedence constraints and if there are available resources—i.e., processors.

The task τ_x releases a DAG-job at time-instant t when it is ready for execution. Then, we assume that all jobs in V_x become ready for simultaneous execution, subject only to the precedence constraints. We assume that the task may release an unbounded sequence of ready DAG-jobs; all $|V_x|$ jobs that are released at some time-instant t must finish the execution before a given time instant $t + D_x$. A minimum interval of duration T_x must pass among consecutive releases of DAG-jobs.

2.2 Properties

Considering the topology of the model, several features can be noticed.

- A *chain* λ in DAG task τ_x is a sequence of vertices $v_1, v_2, \dots, v_k \in V_x$ such that (v_i, v_{i+1}) is an edge in G_x . The length of this chain is characterized by the sum of all the WCETs of all the vertices in the chain: $\sum_{i=1}^k C_i$. The length of the *longest chain* in the DAG G_x is characterized by L_x .
- The *volume* of a DAG is the total WCET of the subtasks of τ_x , namely $vol_x = \sum_{v_i \in V_x} C_i$.
- The *density* of a DAG task is defined as $\delta_x = \frac{L_x}{D_x}$.
- We denote by $U_x = \frac{vol_x}{T_x}$ the *utilization* of a DAG task.

Example 2.1 Let us consider the DAG-task depicted in Fig. 1. For τ_x the longest chain is $\{v_0, v_2, v_4, v_5\}$ and $L_x = 9$. Its volume is $vol_x = 15$, its density $\delta_x = \frac{9}{10} = 0.9$, and its utilization $U_x = \frac{15}{16} = 0.94$.

For each vertex $v_i \in V_x$ two sets $Ancst_i$ and $Desc_i$ can be defined. $Ancst_i$ is the set of ancestor subtasks $v_j \in V_x$ of v_i such that there exists a path starting from v_j that reaches v_i ; $Desc_i$ is the collection of descendant subtasks $v_k \in V_x$ of v_i so as exists a path starting from v_i that reaches v_k .

2.3 Task set of DAGs

A DAG sporadic task system (or task set) Γ is a set of n DAG tasks.

For the task set Γ we denote (i) its maximum density $\delta_{max}(\Gamma)$ to be the largest density of any task in Γ , i.e. $\delta_{max}(\Gamma) = \max_{\tau_x \in \Gamma} \{\delta_x\}$; (ii) its utilization $U(\Gamma)$ to be the sum of the utilizations of all the tasks in τ , i.e. $U(\Gamma) = \sum_{\tau_x \in \Gamma} U_x$; and (iii) its hyper-period $HP(\Gamma)$ to be the least common multiple of the period parameters of all the tasks in Γ , i.e. $HP(\Gamma) = lcm_{\tau_x \in \Gamma} \{T_x\}$.

In a DAG task set Γ each task τ_x is characterized by a priority P_x based on the schedulability algorithm chosen. It should be noted that the use of task priorities assumes a fixed-priority scheduling algorithm, not GEDF. Given the task set and the priorities four sets can be defined, namely $hp(\tau_x)$ ($hep(\tau_x)$) that comprises all the tasks with higher priority (higher or equal priority) w.r.t. τ_x and $lp(\tau_x)$ ($lep(\tau_x)$) that comprises all the tasks with lower priority (lower or equal priority) w.r.t. τ_x .

2.4 Multiprocessor scheduling strategy

When dealing with multiprocessors a strategy on how to map tasks on cores should be chosen.

A *global* scheduler supports the migration of the jobs of the system among the cores during their execution. It is known (Baruah et al. 2015a) that this policy

leads to lower average response time and automatic load balancing while being easy to implement. However, it suffers many overheads due to inter-core synchronization, migration costs, and lack of cache affinity.

The other widely known approach for scheduling multiprocessor real-time systems is *partitioned* scheduling. According to this approach, each task is assigned statically to one specific processor. Partitioned scheduling has the advantages of being supported by the automotive industry (e.g. AUTOSAR), having isolation among cores, and, moreover, established uniprocessor schedulability analysis techniques can be exploited. The problems of this second method are the likely possibility to have unbalanced loads of the cores and the non-deterministic polynomial-time (NP) hardness of the allocation strategies.

Indeed the allocation problem can be modelled as the bin packing problem, which is known to be NP-hard (Cormen et al. 2009). Therefore, heuristics based on a utilization factor are used, as *First Fit*, that fits a task into a processor by scanning from the beginning of available processors to the end, until the first processor that can accommodate it is found; *Best Fit*, which fits a task into a processor by looking for the processor whose left capacity is closer to the utilization of the task; or *Worst Fit*, that fits a task into a processor by looking for the processor with the greatest left capacity.

2.5 Schedulability

A DAG task τ_x is said to be feasible if the subtasks of all of its jobs respect its deadline, i.e., when its Worst-Case Response Time (WCRT), R_x , is less than or equal to its deadline $R_x \leq D_x$. The WCRT is the maximum amount of time required for a task to complete, considering all possible execution scenarios, including the worst-case execution times of all subtasks and the worst-case inter-task dependencies.

When considering the scheduling of a DAG, the concept of critical chain should be introduced. The *critical chain* λ_x^* of a task τ_x is the chain of vertices of τ_x that leads to its WCRT R_x . It is important to remember that the critical chain does not necessarily correspond to the longest chain.

The problem of testing the feasibility of a given DAG task system is highly intractable. Specifically this problem is NP-hard in the strong sense, even when there is a single DAG task (Ullman 1975). Hence, in the literature, several sufficient schedulability tests, of polynomial or pseudo-polynomial complexity, have been introduced.

2.6 Preemption

The preemption strategy is a key aspect of real-time scheduling, as it could favor urgent tasks and affects schedulability.

Traditionally there are two opposite approaches: Fully Preemptive (FP) and Fully Non-Preemptive (FNP) scheduling strategies (Baruah et al. 2015a). In FP scheduling a job can be interrupted at any time if another job with higher priority arrives, and be resumed to continue only when all higher priority jobs have completed their execution. On the other hand, in FNP scheduling the execution of a job cannot be

interrupted: as it starts it executes until completion. In addition Limited Preemption (LP) has been proposed in the literature (Lukkien et al. 2007; Bertogna and Baruah 2010; Buttazzo et al. 2013) to reduce problems related to preemption and migration-related overheads, while also constraining the amount of blocking. This model combines and generalizes the other two: jobs can be preempted, but only in certain points, otherwise they execute Non-Preemptive Regions (NPRs). In the case of DAG tasks however, there is no clear distinction between FNP and LP, because in this context, both terms refer to the execution of DAG nodes without interruption.

Table 1 reports an overview of all the relevant works hereafter discussed, ordered by year, and categorizes them in terms of:

- *workload*, if the method consider a single DAG τ or a task set Γ ;
- *deadline*, implicit (I), constrained (C) or arbitrary (A);
- *model*, task model used in the work: DAG, Conditional DAG (C-DAG), Heterogeneous (or typed) DAG (H-DAG), Heterogeneous Conditional DAG (HC-DAG);
- *scheduling*, multiprocessors policy: global (G), partitioned (P) or federated (F);
- *preemption*, fully preemptive (FP), fully non preemptive (FNP), limited preemptive (LP);
- *algorithm*, earliest deadline first (EDF), deadline monotonic (DM), fixed task priority (FTP) or specific ones (Hier EDF, DAG-Fluid);

A summary of the notation used in this paper is reported in Table 2.

3 Global DAG scheduling

3.1 Global policy, fully preemptive (G-FP)

Baruah et al. (2012) introduced the sporadic DAG task model and focused on the schedulability of a single DAG task with constrained deadlines, in the context of G-EDF scheduling, with a fully preemptive (FP) policy.

The authors proved that the sporadic DAG feasibility problem is NP-hard in the strong sense. Moreover, they have shown that even identifying the worst-case behavior is not trivial. In fact, a sporadic DAG task can be infeasible on m processors even when its synchronous arrival sequence, in which successive DAG-jobs are released exactly the period T timeunits apart, is feasible.

They proposed two sufficient tests: a polynomial test, specific for relative deadline greater than period (Theorems 3.1, 3.2), from which it is possible to compute the minimum number of needed processors, and a pseudo-polynomial test for arbitrary deadlines.

Theorem 3.1 (Baruah et al. 2012) *Any (single) sporadic DAG task $\tau_x = (G_x, D_x, T_x)$ with $D_x > T_x$ satisfying*

Table 1 Summary of the considered works (“–” means not applicable)

Work	Workload	Deadline	Model	Scheduling	Preemption	Algorithm
Baruah et al. (2012)	τ	A	DAG	G	N.A.	EDF
Bonifaci et al. (2013)	Γ	A	DAG	G	FP	EDF, DM
Li et al. (2013)	Γ	I	DAG	G	FP	EDF
Qamhiieh et al. (2013)	Γ	C	DAG	G	FP	EDF
Baruah (2014)	Γ	C	DAG	G	FP	EDF
Saifullah et al. (2014)	Γ	I	DAG	G	FP, FNP	EDF
Li et al. (2014)	Γ	I	DAG	F	FP	Greedy, EDF
Fonseca et al. (2015)	Γ	C	C-DAG	–	–	–
Baruah et al. (2015b)	Γ	C	C-DAG	G	FP	EDF
Melani et al. (2015)	Γ	C	DAG, C-DAG	G	FP	EDF, FTP
Baruah (2015a)	Γ	C	DAG	F	FP	EDF
Baruah (2015b)	Γ	A	DAG	F	FP	EDF
Parri et al. (2015)	Γ	A	DAG	G	FP	EDF, DM
Fonseca et al. (2016)	Γ	C	DAG	P	FP	FTP
Serrano et al. (2016)	Γ	C	DAG	G	LP	FTP
Yang et al. (2016)	Γ	A	DAG	G	LP	FTP
Pathan et al. (2017)	Γ	C	DAG	G	FP	DM
Fonseca et al. (2017)	Γ	C	DAG	G	FP	DM
Serrano et al. (2017)	Γ	C	DAG	G	LP	FTP
Serrano and Quiñones (2018)	τ	C	H-DAG	G	–	–
Han et al. (2019)	τ	C	H-DAG	G	–	–
Casini et al. (2018a, 2018b)	Γ	C	DAG	P	LP	FTP
He et al. (2019)	Γ	C	DAG	G	FP	EDF, FTP
Fonseca et al. (2019)	Γ	C, A	DAG	G	FP	EDF, FTP
Jiang et al. (2019)	Γ	A	DAG	F	FP	EDF
Yang et al. (2019)	Γ	C	DAG	P	FNP	Hier EDF
Nasri et al. (2019)	Γ	C	DAG	G	LP	FTP, EDF
Guan et al. (2020)	Γ	I	DAG	G	FP	DAG-Fluid
Chang et al. (2020)	τ	C	H-DAG	G	–	–
Zahaf et al. (2020)	Γ	C	HC-DAG	P	FP, FNP	EDF
Baruah (2020)	τ	C	DAG	P	FP	EDF
Casini et al. (2020)	Γ	C	DAG	P	LP	FTP

$$(m-1)\frac{L_x}{D_x} + 2\frac{vol_x}{T_x} \leq m \quad (1)$$

is EDF-schedulable on m unit-speed processors.

Theorem 3.2 (Baruah et al. 2012) Any (single) sporadic DAG task $\tau_x = (G_x, D_x, T_x)$ satisfying the following properties

Table 2 Notation used in this paper

Description	Symbol	Description	Symbol
task x	τ_x	ancestors of v_i	$Ancst_i$
job a of task x	τ_x^a	descendants of v_i	$Desc_i$
vertex i of task x	$\tau_{x,i}$	chain l of τ_x	$\lambda_{x,l}$
relative deadline	D_x	set of chains of τ_x	Π_x
period	T_x	length of longest chain in G_x	L_x or $L(G_x), \sum_{i=1}^k C_i$
worst-case response time	R_x	critical chain of τ_x	λ_x^*
No. of preemption suffered by τ_x	pn_x	volume of G_x	vol_x or $vol(G_x), \sum_{v_i \in V_x} C_i$
priority of τ_x	P_x	utilization	$U_x = vol_x/T_x$
DAG	G_x	density	$\delta_x = L_x/D_x$
edge set	E_x	processor to which v_i is assigned to	p_i
vertices set	V_x	task set	Γ
vertex (or node, or subtask) i	v_i	hyper-period of task set Γ	$HP(\Gamma) = lcm_{\tau_x \in \Gamma} \{T_x\}$
WCET of v_i	C_i	higher priority tasks w.r.t. τ_x	$hp(\tau_x)$
local offset of v_i or EST	O_i or EST_i	lower priority tasks w.r.t. τ_x	$lp(\tau_x)$
local deadline of v_i or LFT	D_i or LFT_i	higher-equal priority tasks w.r.t. τ_x	$hep(\tau_x)$
BCET of v_i	BC_i	lower-equal priority tasks w.r.t. τ_x	$lep(\tau_x)$
immediate predecessors of v_i	$pred_i$		
immediate successors of v_i	$succ_i$		

For clarity, a standardized set of indexing names is adopted throughout the entire papers, that is, $\{i, j, k\}$ denote vertices in a DAG, $\{x, y, z\}$ indicate tasks, and $\{a, b, c\}$ are used for jobs in superscription

1. $L_x \leq \frac{2D_x}{5}$
2. $vol_x \leq \frac{2mT_x}{5}$

is EDF-schedulable on m unit-speed processors.

Bonifaci et al. (2013) addressed the same scheduling problem, extending it to a DAG task set with arbitrary deadlines and considering both fully preemptive G-EDF and Global Deadline Monotonic (G-DM).

They proposed a polynomial sufficient test for the G-EDF case (Theorem 3.3).

Theorem 3.3 (Bonifaci et al. 2013) *Assume a sporadic DAG task system satisfies the following conditions:*

1. $L_x \leq \frac{D_x}{3}, x = 1, 2, \dots, n$
2. *for each $x, x = 1, 2, \dots, n$*

$$\sum_{y: T_y \leq D_x} \frac{vol_y}{T_y} + \sum_{y: T_y > D_x} \frac{vol_y}{D_x} \leq \frac{(m + \frac{1}{2})}{3} \quad (2)$$

Hence, the system is EDF-schedulable on m unit-speed processors.

Moreover, they proposed two polynomial sufficient tests for G-DM, one for arbitrary deadlines (Theorem 3.4) and the other for constrained deadlines (Theorem 3.5), with slightly better guarantees.

Theorem 3.4 (Bonifaci et al. 2013) *Consider a sporadic DAG task system satisfies the next conditions:*

1. $L_x \leq \frac{D_x}{5}, x = 1, 2, \dots, n$
2. *for each $x, x = 1, 2, \dots, n$*

$$\sum_{y: T_y \leq 2D_x} \frac{vol_y}{T_y} + \sum_{y: T_y > 2D_x} \frac{vol_y}{4D_x} \leq \frac{(m + \frac{1}{4})}{5} \quad (3)$$

Then the system is DM-schedulable on m unit-speed processors.

Theorem 3.5 (Bonifaci et al. 2013) *Assume a sporadic DAG task system satisfies the following conditions:*

1. $L_x \leq \frac{D_x}{4}, x = 1, 2, \dots, n$
2. *for each $x, x = 1, 2, \dots, n$*

$$\sum_{y: T_y \leq 2D_x} \frac{vol_y}{T_y} + \sum_{y: T_y > 2D_x} \frac{vol_y}{D_x} \leq \frac{(m + \frac{1}{3})}{4} \quad (4)$$

3. $D_x \leq T_x, x = 1, 2, \dots, n$

Then the system is DM-schedulable on m unit-speed processors.

Li et al. (2013) focused on the schedulability of a DAG task set with G-EDF, and they proposed a sufficient test for tasks with implicit deadlines (Theorem 3.6).

Theorem 3.6 (Li et al. 2013) *G-EDF can successfully schedule an implicit deadlines task set Γ of n DAG tasks, if the following conditions are satisfied:*

1. $D_x = T_x, x = 1, 2, \dots, n$
2. $U(\Gamma) \leq \frac{m}{(4 - \frac{2}{m})}$
3. $L_x \leq \frac{1}{(4 - \frac{2}{m})} T_x, x = 1, 2, \dots, n$

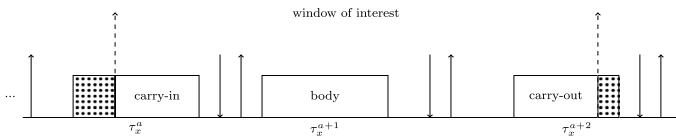


Fig. 2 Carry-in, body and carry-out jobs

Qamhie et al. (2013) later on, proposed another sufficient schedulability test for G-EDF of sporadic DAG task sets with constrained deadlines.

The authors analyzed DAG tasks by assuming their internal structures and providing a tighter bound on the workload and interference analysis. They demonstrated that omitting the structure of the DAGs results in a pessimistic analysis. Their approach is based on assigning a local offset and local deadline for each subtask within the DAG.

Considering a DAG task τ_x , a *local offset* O_i of its vertex v_i is defined as the earliest release time with respect to the activation of τ_x in which vertex v_i may be ready. This is computed as the length of the longest path from the starting vertex of τ_x to v_i . The computation of the local offset of each vertex in the DAG is performed by assuming that the system has an infinite number of processors. This local offset is also referred to as the Earliest Starting Time (EST) in literature. To calculate all the local offsets, a depth-first search algorithm, which runs in linear time, can be applied to each DAG task.

On the other hand, a *local deadline* D_i of vertex v_i is defined as the latest possible deadline of v_i , it is computed as $D_i = D_x - R(v_i)$, where $R(v_i)$ is the length of the longest execution path from a successor of vertex v_i to the ending vertex in the DAG, it does not include the WCET of v_i .

The local deadline is also known in the literature as Latest Finishing Time (LFT). If a vertex v_i misses its local deadline D_i at time t , it guarantees that the DAG τ_x will miss its deadline, even if $t \leq D_x$.

Moreover, for their schedulability test, they improved the analysis of interference between different tasks by considering the roles of body and carry-in jobs.

To better explain these concepts we have to refer to an interval (or window) of interest $[a, b]$ of length t , then the contributions of jobs are typically considered in three separate categories: (i) *carry-in*, that is the contribution of at most one job, which has its release time earlier than a and its deadline in $[a, b]$; (ii) *body*, that is the contribution of jobs with both release times and deadlines in $[a, b]$; (iii) *carry-out*, that is the contribution of at most one job, which has its release time in $[a, b]$ and its deadline later than b . A graphical representation is depicted in Fig. 2.

The derived test is presented in Theorem 3.7.

Theorem 3.7 (Qamhie et al. 2013) *A DAG task set Γ is G-EDF schedulable on m processors of unit speed if:*

$$\begin{aligned}
& \forall x \in \{1, \dots, n\} \\
& \sum_{y=1}^n \sum_{v_i \in V_y} \max(0, \left\lceil \frac{D_x - D_i}{T_y} \right\rceil + 1) \times C_i + \\
& \sum_{y=1, y \neq x}^n \sum_{v_i \in V_y} \min(C_i, \max(0, D_i)) \leq D_x \leq T_x
\end{aligned} \tag{5}$$

Considering the left part of the inequality in Eq. 5, the first member is a Demand Bound Function (DBF, Baruah et al. 1990) and represents the interference of body jobs; while the second denotes the interference of carry-in jobs. The test is sustainable for sporadic DAGs with reduced execution requirements and later arrival times, but it is not sustainable for larger relative deadlines. Sustainability in this context, as formalized in Baruah and Burns (2006) and Burns et al. (2008), refers to the expectation that a system that is schedulable under its worst-case specifications should remain schedulable when its real behavior is “better” than the worst-case scenario.

Baruah (2014) later improved the test for constrained deadline systems introduced by Bonifaci et al. (2013), proposing a test that dominates the previous solution. To do so, the author exploits the concept of *work function*, introduced by Bonifaci et al. (2013), and also later explained and generalized by Baruah et al. (2015a).

Let τ_x be a sporadic DAG task, and s any positive real number ≤ 1 . Let J denote any collection of jobs spawned by the task τ_x . For a given interval I , $work(J, I, s)$ denotes the total execution within the interval I in the schedule $S_\infty(J, s)$ obtained by allocating a speed- s processor to each job the instant it is ready to execute, of jobs with deadlines that fall within I . For any positive integer t , $work(J, t, s)$ defines the maximum value $work(J, I, s)$ can take, over any interval I of length equal to t . Then, let $work(\tau_x, t, s)$ characterize the maximum value of $work(J, t, s)$, over all job sequences J that may be generated by the sporadic DAG task τ_x . Finally, let $work(\Gamma, t, s)$ denote the sum $\sum_{\tau_x \in \Gamma} work(\tau_x, t, s)$. Such definition allows to extend the work function from individual tasks to a DAG task system Γ .

Theorem 3.8 (Baruah 2014) *Let σ denote any constant < 1 . Sporadic DAG task system Γ is G-EDF schedulable on m unit-speed processors if $\delta_{\max}(\Gamma) \leq \sigma$, and*

$$work(\Gamma, t, \sigma) \leq (m - (m - 1)\sigma) \times t \tag{6}$$

for all values of $t \geq 0$.

To demonstrate that a given τ is EDF-schedulable upon m unit-speed processors, it suffices (according to Theorem 3.8), to produce a witness to this fact: a value for σ such that Eq. 6 holds for all $t \geq 0$. The schedulability test presented by Bonifaci et al. (2013) essentially reduces to determining a specific value of σ , namely $\sigma = m/(2m - 1)$. On the contrary, the pseudo-polynomial algorithm derived by Baruah correctly identifies all task systems for which any σ would cause Eq. 6 to evaluate t to true for all $t \geq 0$. Therefore, this test dominated the one by Bonifaci et al. (2013), which is also proven by the author.

Qamhie and Midonnet (2014) analyzed two DAG scheduling methods used in literature, i.e. (i) the parallel scheduling method and (ii) the stretching scheduling method. In the former, common scheduling algorithms are used for scheduling parallel tasks (Baruah et al. 2012; Bonifaci et al. 2013; Li et al. 2013; Baruah 2014); in the latter DAG tasks are transformed into independent sequential tasks with intermediate deadlines and offsets that execute on multiprocessor systems (Qamhie et al. 2013).

The authors considered both methods and showed that they are not equivalent regarding schedulability in the case of G-DM and G-EDF scheduling algorithms, focusing on the implicit deadline case. They concluded that (i) no one dominates the other; (ii) the stretching method is more adapted to fixed task priority assignment algorithm such as DM; (iii) parallel scheduling method achieves better performance in the case of preemptive G-EDF scheduling algorithm, being consequently more convenient to the FTP assignment scheduling algorithms.

Melani et al. (2015) using different global scheduling algorithms, derived efficient methods to compute an upper-bound on the response-time of each DAG task.

A simple but pessimistic WCRT bound R_x for a DAG-task G_x was already proposed by Graham (1969) (Theorem 3.9).

Theorem 3.9 (Graham 1969) *A safe WCRT bound for a DAG G_x on a multiprocessor can be computed as:*

$$R_x = L_x + \frac{1}{m}(vol_x - L_x) \quad (7)$$

where L_x is the length of the longest path in G_x ; vol_x is the total execution time of all vertices in G_x ; and m is the number of cores.

However, Graham bound does not take into account inter-task interference. The authors focused on finding an upper bound on the inter-task interference, considering carry-in, body, and carry-out, and used the Graham bound for intra-task interference.

They derived an upper-bound on the workload of an interfering task τ_y in a window of length t :

$$\mathcal{W}_y(t) = \left\lfloor \frac{t + R_y - vol_y/m}{T_y} \right\rfloor vol_y + \min(vol_y, m \cdot ((t + R_y - vol_y/m) \bmod T_y)). \quad (8)$$

where the first part of the equation represents the upper bound on carry-in and body jobs, the second the one on carry-out. Moreover, they derived an upper-bound on the interfering workload of a task τ_y on a task τ_x with G-EDF:

$$\mathcal{I}_{y,x} = \left(\left\lfloor \frac{D_x - D_y}{T_y} \right\rfloor + 1 \right) vol_y + \min(vol_y, m \cdot \max(0, D_x \bmod T_y - D_y + R_y)) \quad (9)$$

as before, the first term accounts for carry-in and body, the second for carry-out. Finally, they derived a schedulability test based on Response Time Analysis (RTA) both for G-EDF and global Fixed-Priority (G-FTP) algorithms (Theorem 3.10).

Theorem 3.10 (Melani et al. 2015) *Given a task set Γ globally scheduled on m cores, an upper-bound R_x^{ub} on the response-time of a task τ_x can be derived by the fixed-point iteration of the following expression, starting with $R_x^{ub} = L_x$:*

$$R_x^{ub} \leftarrow L_x + \frac{1}{m}(\text{vol}_x - L_x) + \left\lceil \frac{1}{m} \sum_{y \neq x} \mathcal{X}_y^{ALG} \right\rceil \quad (10)$$

With global FP:

$$\mathcal{X}_y^{ALG} = \mathcal{X}_y^{FTP} = \begin{cases} \mathcal{W}_y(R_x^{ub}), & \forall y < x \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

When using global EDF:

$$\mathcal{X}_y^{ALG} = \mathcal{X}_y^{EDF} = \min \{ \mathcal{W}_y(R_x^{ub}), \mathcal{I}_{y,x} \} \quad (12)$$

For any work-conserving scheduler

$$\mathcal{X}_y^{ALG} = \mathcal{W}_y(R_x^{ub}) \quad (13)$$

Parri et al. (2015) proposed polynomial and pseudo-polynomial time schedulability tests, based on RTA, for G-DM and G-EDF scheduling of sporadic DAG tasks with arbitrary vertex utilization and arbitrary deadlines.

The proposed method dominates the one proposed by Bonifaci et al. (2013) in terms of both schedulability ratio and running time, however, it is more complex and less straightforward to apply and implement.

Pathan et al. (2017) were the first to propose a two-level preemptive G-FTP for constrained deadlines: specifically, a task-level scheduler decides the ready task with higher-priority and then a subtask-level scheduler chooses its highest-priority subtask for the execution. The GFP scheduling algorithm enables a subtask to execute on any core of a multicore processor, even if it resumes execution after being interrupted by a higher-priority subtask allowing for better resource utilization and improved schedulability.

In the task-level priority assignment phase, DM priority sorting is used to assign task priorities. The set of tasks having higher fixed priorities than that of task τ_x is denoted by $hp(\tau_x)$. In the subtask-level priority assignment phase, the fixed priorities to the subtasks of τ_x are designated based on a topological order of the subtasks of τ_x .

Then they proposed new methods to compute both inter and intra-task interference and derive a new schedulability test which reduces the pessimism of the analysis by Melani et al. (2015).

The total intra-task interfering workload for a subtask v_i of τ_x , denoted by $\mathcal{W}_{x,i}^{intra}$, is due to the subtasks in set $\mathcal{S}_{x,i}$ and is computed as:

$$\sum_{v_j \in \mathcal{S}_{x,i}} \min \left\{ C_j, \max \left\{ 0, R_{x,j} - \max_{v_k \in Ancst_{x,i}} R_{x,k} \right\} \right\} \quad (14)$$

where $Ancst_{x,i}$ is the set of all the ancestors of v_i and $\mathcal{S}_{x,i}$ comprises all the higher-priority subtask of v_i that are not its ancestors. This formulation is possible because the response times of subtasks $R_{x,j}$ are computed in topological order, therefore all the ancestors will come before their successors.

The inter-task interfering workload $\mathcal{W}_{x,i}(t)$ of all the tasks with higher priority than τ_x ($hp(\tau_x)$), within the problem window t of v_i is:

$$\mathcal{W}_{x,i}^{inter}(t) = \sum_{\tau_y \in hp(\tau_x)} \left(CR_y(t_{cin}) + \left\lfloor \frac{\mathcal{X}_y(t)}{T_y} \right\rfloor \cdot W_y + W_y \right) \quad (15)$$

The first term inside the summation accounts for the carry-in; t_{cin} is the length of the interval inside the problem window where the carry-in job execute, $CR_y(t_{cin})$ is the maximum inter-task interfering workload of the carry-in job of τ_y in t_{cin} . $CR_y(t_{cin})$ is computed considering the work performed by the subtasks of the carry-in job of task $\tau_y \in hp(\tau_x)$. The second term accounts for the body job, $\mathcal{X}_y(t)$ is the length of the interval in which body jobs interfere; and the last term accounts for the carry-out. We refer to the paper for details on how to compute it.

Theorem 3.11 (Pathan et al. 2017) *The response time $R_{x,i}$ of subtask v_i of a DAG task τ_x can be found starting from $R_{x,i}^{(0)} = C_i$ and using the fixed-point recurrence:*

$$R_{x,i}^{(t+1)} \leftarrow \max_{v_j \in Ancst_{x,i}} R_{x,j} + \frac{\mathcal{W}_{x,i}^{intra} + \mathcal{W}_{x,i}^{inter}(R_{x,i}^{(t)})}{m} + C_i \quad (16)$$

The first term on the right-hand side of Eq. 16 characterizes the latest time when subtask v_i becomes ready for execution, the second term denotes the interference on subtask v_i in a problem window of size $R_{x,i}^{(t)}$ and the third term C_i denotes the WCET of subtask v_i .

Theorem 3.12 (Pathan et al. 2017) *Given the response time $R_{x,i}$ of each subtask v_i of task τ_x , the response time R_x of the task can be computed based as*

$$R_x = \max_{v_i \in V_x} \{R_{x,i}\} = R_{x,sink} \quad (17)$$

All the tasks in set $\Gamma = \{\tau_1, \tau_2, \dots, \tau_n\}$ meet their deadlines if $R_x \leq D_x$ for $x = 1, 2, \dots, n$.

Finally the authors theoretically proved that the response-time test in Eq. 16 dominates the response-time test in Eq. 10 with $\mathcal{X}_y^{ALG} = \mathcal{X}_y^{FTP}$.

Fonseca et al. (2017) also presented an RTA technique based on the concept of problem window for the G-FTP scheduling of sporadic DAG tasks.

Authors proposed an algorithm to calculate a tighter upper bound on the intra-task interference, with improved carry-out and carry-in workload computation, from which they derived a new schedulability test (Theorem 3.13).

The analysis considers the topology of the DAG task, based on the concept of workload distribution, both for carry-in and carry-out. To compute the carry-out workload, the DAG is converted in a Nested Fork-Join (NFJ) DAG and then in a sequential-parallel decomposition binary tree (SPTree) (Valdes et al. 1979). Then the authors derived an algorithm to account for the whole interfering workload $\mathcal{W}_y(\Delta)$ for a time interval Δ , combining the carry-in and carry-out contributions.

Theorem 3.13 (Fonseca et al. 2017) *A task τ_x is schedulable under G-FTP iff $R_x \leq D_x$ where R_x is the smallest $\Delta > 0$ to satisfy*

$$\Delta = L_x + \frac{1}{m}(\text{vol}_x - L_x) + \frac{1}{m} \sum_{\tau_y \in \text{hp}(\tau_x)} \mathcal{W}_y(\Delta) \quad (18)$$

The task set is said to be schedulable if all tasks are schedulable. This can be verified by computing Eq. 18 for each task $\tau_x \in \Gamma$, starting from the highest priority task and proceeding in decreasing order of priorities.

The authors showed that the proposed test empirically dominates Eq. 16 Pathan et al. (2017).

To assign priorities to vertices, He et al. (2019) propose a priority assignment algorithm, such that the bound of the response time is decreased as much as possible.

Thus, the authors proposed prioritized list scheduling, which is work conserving and preemptive.

The work is based on the premise that the selection of suitable vertices for execution influences the response time. Essentially, one might prioritize vertices along the longest path for execution to get a reduced response time. The critical path is subject to how the DAG is really scheduled, that is, the critical path of a DAG may be different in distinct execution instances of the DAG.

Theorem 3.14 (He et al. 2019)

The response time R_x of a DAG task τ_x with constrained deadline scheduled by prioritized list scheduling on a platform with m cores can be bounded by

$$R_x \leq \max_{\lambda \in \Pi_x} \left\{ \sum_{v_i \in \lambda} C_i + \frac{\sum_{v_i \in \lambda} I(v_i)}{m} \right\} \quad (19)$$

where Π_x is the set of all complete paths of the DAG G_x and $I(v_i)$ is the interference set of a vertex $v_i \in V_x$ defined as

$$I(v_i) = \{v_j \in V_x \setminus \{v_i\} \mid v_j \notin \text{Ancest}_i \wedge v_j \notin \text{Desc}_i \wedge \text{prio}(v_j) \leq \text{prio}(v_i)\} \quad (20)$$

where a complete path is defined as a path starting from the single source vertex and ending at the single sink vertex, while $\text{prio}(v)$ is the priority assigned to a vertex v . Equation 19 dominates the classic bound in Eq. 7 Graham (1969). However, the number of paths in a DAG can be exponential in the size of the DAG. So it is impractical to enumerate all the paths to compute the response time bound. Therefore, the authors used dynamic programming to solve the problem, deriving an algorithm with complexity $O(|V| + |E|)$.

At task level, the scheduling algorithm is the same to the proposed by Melani et al. (2015), which can be any global work-conserving scheduler, such as RM or EDF. At the vertex level, the vertices within a task are scheduled using a prioritized list scheduling. The authors proposed an algorithm that first priorities vertices based on the topological order, and later assign higher priorities to vertices belonging to longer paths.

Theorem 3.15 (He et al. 2019) *For a constrained deadlines DAG task set Γ scheduled by global prioritized list scheduling on a platform with m cores, a bound R_x on the response time of a task τ_x can be derived by the fixed-point iteration of the following expression, starting with $R_x = L_x$*

$$R_x \leftarrow \max_{\lambda \in \Pi_x} \left\{ \sum_{v_i \in \lambda} C_i + \frac{\sum_{v_j \in \lambda} t(v_j) C_i}{m} \right\} + \frac{1}{m} \sum_{y \neq x} \mathcal{X}_y^{\text{ALG}} \quad (21)$$

where $\mathcal{X}_y^{\text{ALG}}$ is the one from Melani et al. (2015) (Theorem 3.10).

The authors showed that Theorem 3.15 dominates Theorem 3.10 Melani et al. (2015) both theoretically and empirically.

Fonseca et al. (2019) studied again the RTA problem for DAG task set on multiprocessor with global policy, and propose two methods to determine upper-bounds that are less pessimistic on the workload produced by the carry-out and carry-in jobs of the interfering tasks. They used the same formulation as Theorem 3.13 Fonseca et al. (2017) but they proposed new algorithms to compute $\mathcal{W}_y(\Delta)$, both for constrained and arbitrary deadlines.

They compared their solution with Melani et al. (2015) and Parri et al. (2015) and they empirically dominated the first method. However, the complexity of the method and the latency of the tests are not evaluated.

Dong and Liu (2019) introduced novel scheduling and analysis techniques for sporadic DAG tasks on multiprocessors under G-FP-EDF scheduling, which aim to address the challenge of complex intra-DAG precedence constraints that can cause pessimistic schedulability loss in the DAG model. Their method is based on two major components: the Lazy-Cpath policy for determining the execution order of subtasks within a DAG, and a new DAG-specific interval-based analysis

technique. The authors perform several experiments on variants of the proposed methods, however they do not compare with any of the SOTA solutions.

3.2 Global policy, limited preemption (G-LP)

Saifullah et al. (2014) considered the problem of G-EDF within the limited preemptive policy, which means that nodes are not preemptive, but the DAG task can be preempted at node boundaries.

Authors proposed a technique to decompose the DAG task. Upon decomposition, each node of a DAG becomes an individual sequential task with its own deadline and offset, and with a WCET equal to the node's execution requirement.

After the decomposition, they applied literature results both for the fully preemptive (Baruah 2007) and limited preemptive (Baruah 2006) G-EDF for constrained deadline sporadic sequential task set.

Serrano et al. (2016) also focused on limited preemption, combining the results of Melani et al. (2015) and Thekkilakattil et al. (2015).

According to the limited preemption model, tasks are not only interfered by higher-priority tasks, but are also interfered by already running lower-priority tasks whose execution has not yet reached a preemption point, and so cannot be suspended. There are two possible scenarios: (i) in the *eager* approach a high-priority job can preempt the first lower priority job that reaches a preemption point; (ii) in the *lazy* approach a high priority job has to wait until the lowest priority job in execution reaches its preemption point to preempt it. The higher priority interference I_x^{hp} is computed as in Eq. 11 (Melani et al. 2015). The lower priority interference considering G-FTP scheduling with eager preemptions is:

$$I_x^{lp} = \Delta_x^m + pn_x \times \Delta_x^{m-1} \quad (22)$$

where pn_x is an upper-bound on the number of preemptions experienced by τ_x , and Δ_x^m and Δ_x^{m-1} are upper-bounds on the lower-priority interference on the first NPR and the p^{th} NPRs of task τ_x , respectively. A straightforward way to derive the lower priority interference is to consider the m and $m - 1$ largest NPRs among all lower-priority tasks as:

$$\Delta_x^m = \sum_{\tau_y \in lp(\tau_x)}^m \max \left(\max_{\forall v_i \in V_y}^m C_i \right) \quad (23)$$

$$\Delta_x^{m-1} = \sum_{\tau_y \in lp(\tau_x)}^{m-1} \max \left(\max_{\forall v_i \in V_y}^{m-1} C_i \right) \quad (24)$$

Theorem 3.16 (Serrano et al. 2016) *Given a task set Γ scheduled on m cores with G-FP and limited preemption, an upper-bound R_x^{ub} on the response-time of a task τ_x can be derived in pseudo-polynomial time by the fixed-point iteration of the following expression, starting with $R_x^{ub} = L_x$:*

$$R_x^{ub} \leftarrow L_x + \frac{1}{m}(vol_x - L_x) + \left\lfloor \frac{1}{m}(I_x^{lp} + I_x^{hp}) \right\rfloor \quad (25)$$

Notwithstanding its simplicity, this strategy is pessimistic because it mainly assumes that the largest m and $m - 1$ NPRs can execute in parallel, disregarding of the precedence constraints defined in the DAG. Moreover, to calculate the blocking impact of the largest Parallel NPRs, authors, presented a pseudo polynomial Integer Linear Programming (ILP) method, which is less pessimistic and dominates over the previous one.

Additionally, Serrano et al. (2017) investigated the results for G-FTP with limited preemption (LP), analyzing the eager and lazy cases, using the scheduling techniques derived in their previous work (Serrano et al. 2016).

The authors concluded that LP-eager outperforms LP-lazy, despite a large number of priority inversions are considered in the analysis, and a considerable number of preemptions are noticed at system deployment. Thus, the LP lazy scheduling approach has been demonstrated to be a inefficient scheduling technique when DAG-task sets are under consideration, being consequently not appropriate for parallel execution.

Nasri et al. (2019) provided a schedulability analysis for global limited-preemptive earliest-deadline first (G-LP-EDF) or fixed-priority (G-LP-FTP) scheduling.

The analysis builds up a schedule-abstraction graph that abstracts all the possible orderings of job dispatch times resulting from the underlying scheduling policy, based on which the authors derived bounds on the best- and worst-case response time of each job. Because jobs experience both release and execution time jitter, exponentially many execution scenarios exist, and consequently the exact finishing time of each job cannot be known beforehand. For this reason, they considered an interval $[EFT_x, LFT_x]$ in which a job τ_x^a will finish. The authors proposed a novel algorithm to compute EST and LST of the jobs.

The schedule-abstraction graph is built iteratively in two different stages: expansion and merging. The expansion stage extends—one of—the shortest path(s) λ in the graph by considering all the jobs that can likely be dispatched next in the job-dispatch sequence represented by λ . The merge stage slows down the growth of the graph by merging—whenever possible—the terminal vertices of paths that have the same collection of dispatched jobs. The searching phase ends when there are no vertex left to expand, i.e., when all paths represent a valid schedule of all jobs considered, implying that all possible schedules have been explored.

Lastly, the proposed analysis dominates (Serrano et al. 2016) and many other methods for sequential tasks. Nevertheless, it does not scale to highly parallel DAG tasks or systems with a large number of processors (e.g., more than 64). The code was made publicly available.¹

¹ <https://github.com/brandenburg/np-schedulability-analysis>.

3.3 Exact tests

As already mentioned, testing the feasibility of a DAG task system is a NP-hard problem in the strong sense, and there exist no methods to solve it in polynomial time. Exact tests are however valuable, even if they can be applied only on small examples due to their computational costs, to be compared with sufficient tests for understanding their goodness. There are not many works on finding exact results for single DAG tasks, no one for a DAG task sets. In the following, the most related ones are reported.

Burmyakov et al. (2015) proposed an exact schedulability test for sporadic real-time tasks with constrained deadlines, for G-FP.

The authors employed a set of techniques that cut down the state space of exploration of the analysis. They extended the prior work by Bonifaci et al. (2013) and made their C++ code publicly available.²

The greatest limitation is that the proposed method can evaluate DAG tasks featuring only one path.

Yalcinkaya et al. (2019) presented the first exact schedulability test for LP (and FNP) self-suspending constrained real-time tasks scheduled upon a uniprocessor or multiprocessor platform (under a G-FP scheduling policy). The authors mapped the schedulability problem to the reachability problem in timed automata (TA), using TA extensions available in UPPAAL.³

Moreover, this work provides the first exact baseline against which sufficient schedulability tests for self-suspensions and limited-preemptive models can be compared.

Sun et al. (2020) studied the exact scheduling of a single non-recurrent DAG task running on a multi-processor architecture under the list scheduling algorithm in a non-preemptive manner.

The main idea behind the method proposed is derived from the traditional scheduling theory, which formulates the RTA problem into an optimization problem. The authors devised and implemented the analysis by using Satisfaction Modular Techniques (SMT), and formulate the WCRT analysis problem for a DAG task under the list scheduling algorithm upon the multi-processor platform into an SMT program.

The number of constraints considered in the SMT model is bounded by $O(|V||E|)$. The authors proved that using the proposed constraints in their SMT, can specifically solve the WCRT of DAGs under the list scheduling upon m processors.

4 Partitioned DAG scheduling

As previously mentioned, when dealing with partitioned scheduling, before solving the scheduling problem, one should first solve the problem of nodes allocation onto the available cores (also NP-hard).

² www.cister.isep.ipp.pt/docs/CISTER-TR-150503.

³ <https://www.uppaal.com/>.

A common practice is to assume the nodes already allocated, or allocate them with a simple heuristic as First-Fit, Best-Fit or Worst-Fit. These methods offer ways to decide the order in which the cores are chosen.

However, for DAG task, also the way the nodes are allocated matters. Straightforward ordering could be by decreasing node utilization or density. In this work, we propose a new ordering, called Decreasing Remaining Utilization (DRU), in which nodes are ordered by their decreasing remaining utilization. The remaining utilization is computed for each node as the sum of the utilization of its successors. Several allocation heuristics are discussed and analyzed in Sect. 7.

In the following, instead, partitioned schedulability tests are discussed.

Fonseca et al. (2016) focused on the partitioned scheduling for fixed-priority (P-FTP) scheduling of DAG tasks on multiprocessors, assuming the partitioning to be given, fully preemptive policy and constrained deadlines. The authors modeled a partitioned DAG task as a collection of self-suspending tasks and present a novel algorithm to traverse a DAG characterizing the worst-case scheduling scenario.

In their setting, each vertex of a DAG task is assigned to a specific core. Then, different vertices of the DAG can execute in parallel on the multiprocessor platform—but without migration. Therefore, each subtask is characterized not only by its WCET, but also the core to which it is assigned p_i ; that is, $v_i = (C_i, p_i)$. They denoted by $\lambda_{x,l}$ a path l of task τ_x and by $|\Pi_x|$ the number of all the possible different paths $\lambda_{x,l} \in \Pi_x$.

Theorem 4.1 (Fonseca et al. 2016) *The worst-case response time of a path $\lambda_{x,l}$ of a partitioned DAG task τ_i is given by*

$$R(\lambda_{x,l}) = \text{len}(\lambda_{x,l}) + I_x(\lambda_{x,l}) + \sum_{\forall \tau_y \in hp(\tau_x)} I_y(\lambda_{x,l}) \quad (26)$$

where $\text{len}(\lambda_{x,l})$ is the length of the path $\lambda_{x,l}$; $I_x(\lambda_{x,l})$ is the self-interference and $\sum_{\forall \tau_y \in hp(\tau_x)} I_y(\lambda_{x,l})$ is the inter-task interference.

Consequently, the worst-case response time of a DAG task τ_x is given by

$$R_x = \max_{l=1}^{|\Pi_x|} R(\lambda_{x,l}) \quad (27)$$

A DAG task τ_x is deemed schedulable if $R_x \leq D_x$.

A path λ can be specified as a collection of sporadic self-suspending tasks, one for each core reached by the path, that is, $\forall p \in \text{proc}(\lambda)$. Specifically, the subtasks of λ allocated to the current processor under analysis are treated as execution regions and the response time of all the remaining subtasks as suspension regions. Therefore, the problem of calculating the response time of λ on a multiprocessor platform becomes equivalent to the analysis of $|\text{proc}(\lambda)|$ self-suspending tasks in a single-core system. Nevertheless, unlike former studies, the length of the suspension regions is unknown a priori as they are computations to be executed on distinct processors.

The authors proposed a novel algorithm that recursively breaks down a path into smaller paths, creating consequently a tree of subpaths that represent self-suspending tasks. The output tree express the hierarchy of dependencies. The corresponding task has no suspension regions when a leaf is reached—meaning that it is a sequential task –, thus its WCRT does not rely upon anything else besides the interfering workload on that processor and can be rapidly processed. Then, the resulting values are back-propagated to the self-suspending tasks on the upper levels, in this way, their suspension time is no longer unknown.

Then, the WCRT of a single subtask is computed by just including the self-interfering workload to the long-established equation for fixed-priority sequential tasks in single-core systems.

The authors in this work also showed how three different SOTA response time analyses (Bletsas 2007; Nelissen et al. 2015) for sporadic self-suspending tasks can be extended to manage both the self-interfering workload and the dependent suspension regions.

Casini et al. (2018a, 2018b) focused again on P-FTP, analyzing the non-preemptive scenario, or better, the limited preemptive case for DAG tasks. They proposed both a new analysis that exploits self-suspending tasks and a partitioning algorithm.

A segmented self-suspending task τ_x^{ss} is characterized by an ordered set of N_x^S execution segments alternated by self-suspensions, both with bounded length and characterized by the tuple

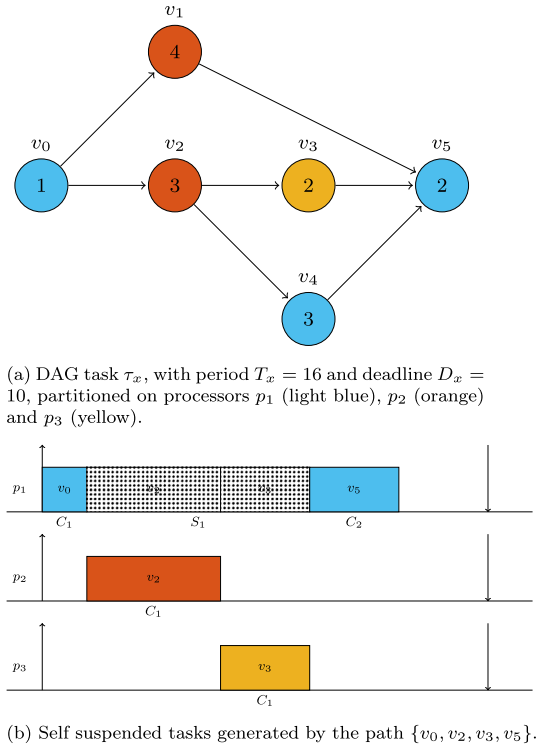
$$\langle C_{x,1}, S_{x,1}, \dots, S_{x,N_x^S-1}, C_{x,N_x^S} \rangle,$$

where $C_{x,i}$ represents the WCET of the i -th execution segment of τ_x^{ss} and $S_{x,i}$ describes the maximum duration of the i -th self-suspension of τ_x^{ss} .

They proposed an algorithm to convert a DAG into a set of segmented self-suspending tasks, based on Fonseca et al. (2016). Specifically, the proposed algorithm considers as input a path $\lambda_{x,l}$ that must begin and end with nodes allocated to the same processor p_i . It returns a self-suspending task modeling the execution of $\lambda_{x,l}$ on p_i , and an upper bound on the total suspension time that the self-suspending task may incur. To better understand, let us consider Fig. 3. Figure 3a represents a DAG partitioned on three processors: $\{v_0, v_4, v_5\}$ are assigned to processor p_1 (light blue), $\{v_1, v_2\}$ are assigned to processor p_2 (orange), and $\{v_3\}$ is assigned to processor p_3 (yellow). Figure 3b shows the analysis of the path $\{v_0, v_2, v_3, v_5\}$: for processor p_2 and p_3 two self-suspended task with a single execution segment are derived; for processor p_1 a single self-suspended task is obtained with two execution segments (represented by v_0 and v_5) and a single suspension region that accounts for v_2 and v_3 . To complete the analysis, the self interference and the high priority tasks interference needs to be accounted; moreover this procedure has to be performed for each path in the DAG.

The authors then proposed two safe response-time bounds for self-suspending non-preemptive tasks.

Fig. 3 Example of conversion of a path in the DAG into self-suspend tasks (Color figure online)



Theorem 4.2 (Casini et al. 2018b) *The response-time of a self-suspending task τ_x^{ss} is bounded by*

$$R_x = R'_x + C_{x,N_x^S} \quad (28)$$

where R'_x is given by the least positive fixed point of the next recursive equation:

$$R'_x = \sum_{i=1}^{N_x^S-1} (C_{x,i} + S_{x,i}) + \sum_{b \in \mathcal{B}_x(N_x^S, R'_x, \bar{\mathbf{R}})} b + I_x(R'_x) \quad (29)$$

that is the sum of (i) the execution of the task, (ii) its maximum blocking time and (iii) the maximum interference experienced.

In the equation $\mathcal{B}_x(N_x^S, R'_x, \bar{\mathbf{R}})$ is the multiset that includes the N_x^S largest elements into $C_i(R'_x, \bar{\mathbf{R}})$, padded with zeros if $|C_i(R'_x, \bar{\mathbf{R}})| \leq N_x^S$, and $\bar{\mathbf{R}}$ is a vector of safe response-time bounds for each execution segment.

Theorem 4.3 (Casini et al. 2018b) *The response-time of the l -th segment of a self-suspending task τ_x^{ss} is bounded by*

$$R_{x,l} = \sum_{i=1}^l C_{x,i} + \sum_{i=1}^{l-1} S_{x,i} + \sum_{b \in \mathcal{B}_x(l, r_{x,l}(\bar{\mathbf{R}}), \bar{\mathbf{R}})} \Delta_x(b, \bar{\mathbf{R}}) \quad (30)$$

where $r_{x,l}(\bar{\mathbf{R}})$ is the latest release time of the l -th segment of τ_x^{ss} and $\Delta_x(b, \bar{\mathbf{R}})$ is the maximum time it can be delayed from its release up to the time it starts executing.

They showed that both Theorem 4.2, at self-suspended task level, and Theorem 4.3, at segment level, offer safe response-time bounds for each task τ_x^{ss} and its execution segments. Therefore the minimum of the two is still a safe response-time bound. They proposed an algorithm to combine the two to implement a schedulability test for segmented non-preemptive self-suspending tasks. The algorithm is then applied to study the response time of a path, including also the self-interference of a DAG task.

Finally, the authors presented an algorithm for partitioning parallel tasks upon the various processors of a multiprocessor architecture. The algorithm uses the analysis presented in the precedent section and is general enough to be integrated with different partitioning heuristics such as Best-Fit, Worst-Fit, First-Fit and different orderings strategies for task selection.

In addition to that, the same authors at the same conference (Casini et al. 2018a) (RTSS 2018), proposed strategies for bounding the worst-case memory space requirement the DAG task set analyzed executing on multiprocessor architectures featuring scratchpad memories. They introduced a feasibility test that certifies whether memories are sufficiently large to include the maximum memory backlog that may be produced by the system. In this work, closed-form bounds and more precise algorithmic strategies are introduced.

Casini et al. (2020) expand upon their prior research (Casini et al. 2018a) by conducting a detailed examination of the memory contention caused by parallel tasks executing on a multiprocessor architecture.

The DAG task is extended to include memory latencies, with edges representing producer-consumer communications between nodes. Each edge $e_{ij} = (m_{ij}, \xi_{ij})$ among vertices v_i and v_j , is associated with a weight m_{ij} , and a worst-case memory access time ξ_{ij} . Specifically, m_{ij} characterizes the number of transactions required to move from global memory, the data produced by node $v_{x,i}$ and consumed by $v_{x,j}$. ξ_{ij} express the maximum amount of time needed to perform m_{ij} requests in isolation, that is, without the contention originated by nodes executing on the other processors. For each communication $e_{ij} \in E_x$, the corresponding data buffer is assigned to a single DRAM bank.

The computation of the nodes follows a three-phase pattern. First, it is performed a copy-in phase to load into the local memory all the data used in global communications. Once the copy-in phase is concluded, the node can execute only accessing the local memory (that is, it cannot experience memory contention). Once the execution of the node is finished, a copy-out phase is performed to store all the data used in global communications in the global DRAM memory. The process concludes with the completion of the copy-out phase.

Inter-task communication between two nodes $v_{x,i} \in V_x, v_{y,j} \in V_y$ belonging to different tasks is handled through the use of dummy nodes and edges. Specifically, if $v_{x,i}$ is producing data for $v_{y,j}$, a dummy node $v_{x,k}$ is added to V_x , and a dummy edge $e(v_{x,i}, v_{x,k})$ is added to E_x to connect $v_{x,i}$ and $v_{x,k}$. Dummy nodes are not executed as they do not possess execution time. The pre-fetching of instructions from global memory and communication among consecutive jobs within the same task can be similarly managed.

The WCET of each vertex is computed as $C_{x,i}^* = R_{x,i}^{IN} + C_{x,i} + C_{x,i}^{OUT}$, where $R_{x,i}^{IN}$ is the upper bound on the response time of the copy-in phase of a node $v_{x,i}$, $C_{x,i}$ is its original WCET and $C_{x,i}^{OUT}$ is the contention-free WCET of the copy-out phase. When the inflated WCETs are available for each node and task, a RTA for parallel tasks under non-preemptive scheduling can be applied, as outlined in the work of Casini et al. (2018b).

4.1 Exact tests

Baruah (2020) made an effort at obtaining an exact algorithm for scheduling a DAG on multiprocessors when processor assignment is specified. The processors are assumed to be preemptive and each processor is considered separately.

The author derived an algorithm for representing the scheduling problem as an ILP, which can then be solved using standard ILP-solvers. He showed that solving even this simple problem turned out to be surprisingly challenging. It required the author to draw upon, and integrate, disparate ideas from Operations Research and real-time scheduling theory (i.e. DBF) to synthesize the ILP. Then he had to fall back on results from real-time scheduling theory (the optimality of preemptive uniprocessor EDF) to synthesize the actual schedule using the individual jobs' start-times and completion times as determined by the solution to the ILP.

The proposed ILP has a computational complexity of $\mathcal{O}(n^3)$, which means that the ILP is of size polynomial in the number of jobs.

5 Other approaches

5.1 Federated scheduling

Li et al. (2014) analyzed for first the problem of federated scheduling of DAG tasks on multiprocessors.

Given a task set Γ , tasks are divided into two disjoint sets: Γ_{high} contains all high-utilization tasks ($\forall \tau_x \in \Gamma | U_x \geq 1$), and Γ_{low} contains all the remaining low-utilization tasks. Considering a high-utilization task τ_x , m_x dedicated cores are assigned to it, m_x is

$$m_x = \left\lceil \frac{vol_x - L_x}{D_x - L_x} \right\rceil \quad (31)$$

m_{high} denotes the total number of cores assigned to high-utilization tasks and is computed as $m_{high} = \sum_{\tau_x \in \Gamma_{high}} m_x$. The remaining cores $m_{low} = m - m_{high}$ are allocated to all the low-utilization tasks.

Theorem 5.1 (Li et al. 2014) *The federated scheduling algorithm admits the task set Γ , if*

1. m_{low} is non-negative;
2. $m_{low} \geq 2 \sum_{\tau_x \in \Gamma_{low}} U_x$.

If the task set is feasible then (i) any work-conserving parallel scheduler can be applied to schedule a high-utilization task τ_x on its assigned m_x cores; (ii) low-utilization tasks are considered and executed as sequential tasks and any multi-core scheduling algorithm can be used to schedule Γ_{low} on m_{low} cores.

Baruah (2015a) introduced a two-phase algorithm for the federated scheduling of any system of constrained-deadline sporadic DAG tasks.

As a first step, the author developed an algorithm for each high-density task (i.e., $\forall \tau_x \in \Gamma | \delta_x \geq 1$), in which the number of processors to be devoted to this task is determined. Then the schedulability is checked with Graham's list scheduling algorithm (Graham 1969).

Then, the remaining low-density tasks are assigned upon the remaining processors. Since any intra-task parallelism cannot be exploited upon a single processor, the internal structure of the DAG can be ignored and each task can be represented in the classic three-parameter sporadic model. An algorithm to partition the tasks on the cores based on DBF (Baruah et al. 1990) is presented. Every shared processor is scheduled using preemptive uniprocessor EDF in run-time.

Baruah (2015b) later on the same year, extended his previous work also for arbitrary deadline systems. The main difference w.r.t the previous work is an extension of Graham's list scheduling algorithm with constraints in order to apply to arbitrary deadlines.

As the authors state, this method is efficiently implementable in time that is polynomial in the representation of the tasks in Γ and the number of processors m .

Jiang et al. (2016) formally proposed the semi-federated scheduling approach for scheduling parallel real-time tasks on multi-core systems. They addressed the issue of resource waste in traditional federated scheduling by allowing heavy tasks with processing capacity requirements slightly larger than an integer value to use only the integer number of dedicated processors, and scheduling the remaining fractional part of their capacity requirement with light tasks on shared processors. The authors demonstrated through experiments that the semi-federated scheduling has

very promising results and can outperform federated scheduling and other existing approaches for scheduling parallel real-time tasks on multi-core systems.

Ueter et al. (2018) introduce a reservation-based federated scheduling approach for parallel real-time tasks with constrained and arbitrary deadlines on homogeneous multi-core processors. Authors extended traditional federated scheduling by allocating reservation servers, which simplifies the scheduling problem for arbitrary-deadline DAG task sets by reducing it to the problem of scheduling arbitrary-deadline sequential task sets. This work also provides a general reservation design for sporadic parallel tasks, which enables the use of any scheduling algorithm and analysis for sequential tasks with arbitrary deadlines on the allocated reservation servers of parallel tasks. The proposed reservation-based federated scheduling algorithms achieve constant speedup compared to any optimal scheduler for arbitrary-deadline DAG task sets.

Jiang et al. (2019) studied the analysis of G-EDF scheduling for DAG parallel tasks with arbitrary deadlines, focusing on the case of $D > T$.

Theorem 5.2 (Jiang et al. 2019) *Any DAG task τ_x is schedulable on m processors when $U_x \leq m$ if at least one of the following conditions holds.*

$$\frac{\text{vol}_x[U_x] + (m-1)L_x}{m} \leq D_x \quad (32)$$

$$\frac{U_x \times L_x}{m - U_x} + \frac{\text{vol}_x + (m-1)L_x}{m} \leq D_x \quad (33)$$

The schedulability tests in Theorem 5.2 can be effortlessly expanded to the analysis of multiple DAG tasks under federated scheduling, where each heavy task ($U_x > 1$) is partitioned into different specific processors and exclusively runs on them while all light tasks ($U_x \leq 1$) divide the rest of the processors together as if they were sequential tasks.

Then the schedulability test for a DAG task set Γ includes two different steps: (i) partition dedicated processors to each heavy task according to Theorem 5.2 if there are enough processors, alternatively, return failure, and (ii) test whether all light tasks are schedulable on the remaining processors treating them as arbitrary-deadline sequential tasks under partitioned EDF, as in the work of Baruah (2015b).

5.2 Other perspectives

Non-standard approaches are those methods that propose their schedulability algorithms and therefore cannot be fairly compared with the rest of the literature. They pertain though to this paper because they try to solve the problem in unconventional ways, offering new points of view.

Qamhieh et al. (2012) studied the global scheduling of n synchronous periodic parallel real-time graphs with implicit deadlines running on m identical processor

system. The feasibility of scheduling is evaluated over the duration of the hyper-period for each set of tasks.

Least Laxity First job priority assignment is adopted to schedule every subtask in the graph in compliance with their laxity while considering the global deadline and period without the need to assign them local ones.

Guo et al. (2017) have been the first authors to study energy aware scheduling of sporadic DAG tasks with implicit deadlines. While meeting all real-time constraints, authors studied how to minimize the platform power consumption by applying task allocation techniques and by finding the best execution scheme. The authors proposed an energy-sub-optimal scheduler adapting the decomposition based framework for federated scheduling. Finally, they derived an approximation algorithm for identifying the cores that can be merged for improving the energy efficiency.

Yang et al. (2019) focused on the scheduling of a DAG task set under partitioned EDF scheduling. Authors proposed a novel method that considers a hierarchical scheduling approach which breaks down the overall scheduling problem into two different parts: (i) scheduling DAG tasks onto virtual processors, (ii) scheduling virtual processors on physical processors. Particularly, each DAG task is partitioned within multiple dedicated virtual processors. Each DAG can be analyzed independently as in federated scheduling with proper the characterization of the resource provided by virtual processor. In contrast, virtual processors are scheduled onto the physical processors at runtime which effectively enables the processor sharing between different DAG tasks.

The idea behind this work is that the hierarchical scheduling approach adopts the advantages of both global scheduling and federated scheduling, and thus achieves better schedulability.

Guan et al. (2020) proposed DAG-Fluid, a real-time scheduling algorithm for DAG task sets. DAG-Fluid is a method that combines task decomposition with a corresponding scheduling algorithm based on fluid scheduling.

For a task τ_x , DAG-Fluid behaves differently based on the value of U_x . If $U_x \leq 1$ the task is considered “light,” and DAG-Fluid transforms τ_x into a non-parallel task where all of its subtasks are required to execute sequentially with a constant execution rate. If $U_x > 1$, DAG-Fluid first transforms the heavy task into different successive segments and then assigns an execution rate to each segment individually. Once rates have been assigned to both light and heavy tasks, the scheduling process is straightforward: whenever the task (or subtask) is ready, it starts executing immediately at the assigned constant execution rate. The offline part of the algorithm has polynomial time complexity, while the online part has linear time complexity.

6 Extensions to DAG task models

Finally, all the extensions to DAG task based model are hereafter discussed, namely (i) Conditional DAGs, (ii) Heterogeneous DAGs and (iii) Heterogeneous Conditional DAGs.

6.1 Conditional DAG tasks

Conditional DAGs are the first proposed extension to the DAG task. The task model is enriched with new types of vertices, namely conditional vertices, that represent conditions in the execution of the task. In this way, constructs as `if-then-else` or `switches` can be represented. The model becomes more expressive but also more complex.

Fonseca et al. (2015) analyzed for first the problem of conditional branches within the DAG model, for scheduling of constrained-deadlines task set. They presented a multi-DAG model where each task is represented by a collection of execution flows. In this sense, each execution flow represents a different execution path throughout the task code and is modeled as a DAG of sub-tasks.

Specifically, the authors introduced a two-step solution that processes a single synchronous DAG of servers for a task modeled by a multi-DAG. This solution demonstrates that these servers can provide the required CPU budget for every execution flow of the task, allowing it to execute entirely while fulfilling its precedence constraints, regardless of the execution flow taken at runtime.

In the multi-DAG model each task τ_x is characterized by a 3-tuple (F_x, T_x, D_x) . Considering control structures within τ_x 's code (such as the “if-then-else” statements), two different jobs of τ_x may execute two different parts of the code, called execution flows, represented by F_x . F_x is the set of execution flows, namely a collection of DAGs.

For each execution flow $F_{l,m}$ of each task τ_x , a synchronous DAG of servers, named as synchronous server graph (SSG) which is denoted by $F_{l,m}^{SSG}$ is derived. Then, authors present an algorithm to merge all the SSGs $F_{l,m}^{SSG}$ created for a task τ_x , into a single synchronous DAG of servers, called “global synchronous server graph” (GSSG) and denoted by F_x^{GSSG} .

If a valid GSSG F_x^{GSSG} is considered schedulable by a schedulability test of a scheduling algorithm A , the task τ_x from which it was derived is also considered schedulable. Thus, the previous results for the DAG task model can be applied, once F_x^{GSSG} is calculated.

Baruah et al. (2015b) formally introduced the conditional sporadic DAG task model—from now on called C-DAG—as an extension to the sporadic DAG task model (Baruah et al. 2012) that is suited to modeling certain conditional control-flow constructs.

Similar to the classic sporadic tasks, every conditional sporadic DAG task τ_x is specified as a 3-tuple (G_x, D_x, T_x) , where $G_x = (V_x, E_x)$ is a DAG that, apart from regular vertices, has also conditional ones. Conditional vertices are special vertices in V_x that are defined in pairs. Let (c_1, c_2) be such a pair in the DAG $G_x = (V_x, E_x)$. Vertex c_1 denotes a part of the code where a conditional expression is examined and, depending on the result of this analysis, control will subsequently flow along particularly one of several different potential paths in the code. It is necessary that all the different paths join again at a common point in the code, characterized by the vertex c_2 .

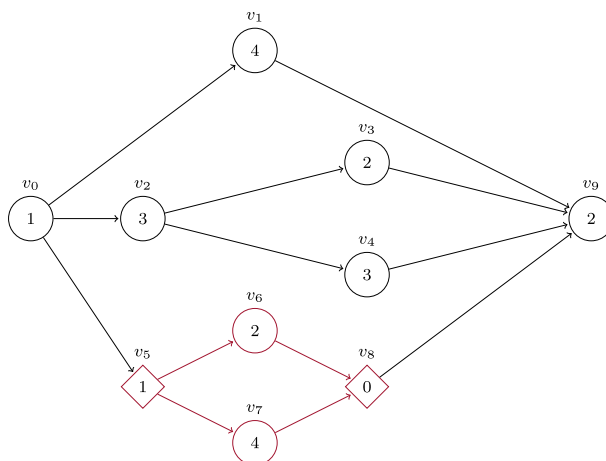


Fig. 4 C-DAG τ_x , with period $T_x = 16$ and deadline $D_x = 10$. (v_5, v_8) is a pair of conditional vertices (diamonds), where v_5 is the entry point and v_8 is the exit point. The whole conditional construct (v_5, v_6, v_7, v_8) is depicted in red (Color figure online)

Edges (v_1, v_2) among pairs of vertices, neither of which are conditional vertices, characterize precedence constraints as in the common sporadic DAG tasks, while edges involving conditional vertices characterize conditional execution of code. Particularly, let (c_1, c_2) denote a defined pair of conditional vertices. After the vertex c_1 completes the execution, one of its successors becomes eligible to be executed; the successor to be executed can not be known beforehand. Vertex c_2 begins the execution on the completion of one of its predecessors.

For each pair (c_1, c_2) of conditional vertices in G_x , we refer to the subgraph of G_x beginning at c_1 and ending at c_2 as a conditional construct in G_x . It is permitted for conditional constructs to be nested: a conditional construct may include more conditional constructs within it.

Let $\mathcal{J}_x$ denote all possible complete set of jobs that comprise a C-DAG τ_x . Each $J \in \mathcal{J}_x$ characterizes a set of jobs obtained by completely executing through the DAG G_x once, considering the conditional branches within it. For each $J \in \mathcal{J}_x$, all the jobs have a common deadline and common release time. Be noted that, $|\mathcal{J}_x|$ may be exponential in the number of vertices in G_x .

In Fig. 4 an example of C-DAG is depicted.

Besides the introduction of this new model, the authors also analyzed the schedulability of C-DAG task sets with constrained-deadline on multiprocessors.

Firstly, the authors proposed a method to compute the *work* function for this new task model, in order to apply the results of Baruah (2014) (Theorem 3.8).

Although its correctness, the method has the same problem as the multi-DAG model of Fonseca et al. (2015): the number of different possible flows of control may be exponential with the dimension of the DAG; the algorithm would then require exponential time.

For this reason, the authors developed a novel transformation technique that transforms conditional sporadic DAG task to a non-conditional in polynomial

time, and tests the system of transformed tasks for G-EDF schedulability using the pseudo-polynomial test provided by Bonifaci et al. (2013).

Melani et al. (2015) at the same time (ECRTS 2015) derived techniques to efficiently compute an upper-bound on the response-time of each C-DAG using different global scheduling algorithms.

The method introduced in Sect. 3.1, was actually developed for C-DAGs. The analysis is exactly the same, except for the computation of: (i) the longest path L_k and (ii) the volume vol_k (called workload W_k for C-DAG) of the DAG. Indeed, when handling C-DAG, those two factors depend on the choices made on the conditional branches. The authors proposed algorithms to efficiently compute those values.

Experiments among randomly generated C-DAG workloads show that the proposed approach improves over previously proposed solutions. The code for the analysis of the proposed method and other several ones has been made public.⁴

Pathan et al. (2017) analyzed G-FP scheduling of DAG (as already introduced in Sect. 3.1) and their analysis can also be applied to C-DAGs. Their method reduces the pessimism of the analysis of Melani et al. (2015) also in the case of conditional DAG tasks.

Sun et al. (2019) considered the problem of G-FP scheduling of C-DAG OpenMP tasks on multiprocessors. The innovative part of their model is that they studied the problem of how to bound their WCRT when non-well-nested branching structures are in the C-DAG.

A C-DAG contains a non-well-nested branching structure when there exists an edge between a regular vertex of one of the conditional construct and a vertex that does not belong to that conditional construct. For example, inserting $e(v_6, v_4)$ in the C-DAG τ_x of Fig. 4 would create a non-well-nested branching structure. This possibility was not taken into account in the previous works.

Starting from the Graham bound (Graham 1969) (Theorem 3.9), they proposed a method to compute the volume and the length of C-DAGs with non-well-nested branching structures.

6.2 Heterogeneous DAG tasks

Heterogeneous, or typed, DAGs are another extension of the DAG task model. The additional feature of this extended DAG task is a new property of the vertices. To take into account the fact that sub-tasks can execute on different types of cores or different engines, each vertex is further described by means of a tag that specifies the kind of computational unit on which it will run onto.

Yang et al. (2016) presented for the first time a DAG-based RTA implemented on heterogeneous multicore platforms.

The compute engines (CE) are clustered in pools, so that all the same kind of CE are grouped in the same pool. The authors focused on non-preemptive G-EDF

⁴ https://retis.sssup.it/~d.casini/resources/DAG_Generator/cptasks.zip.

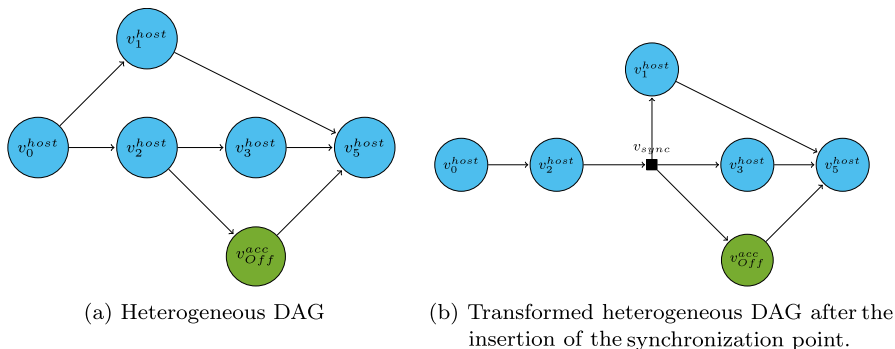


Fig. 5 Example of the heterogeneous-DAG task proposed; before (a) and after (b) the insertion of a synchronization point (black square). Regular nodes that execute on the host are filled in light blue; the single node that executes on the accelerator is filled in green (Color figure online)

scheduling algorithm within each CE pool. They introduced the offset-based independent task (obi-task) model and an algorithm to convert DAG tasks to obi-tasks.

Serrano and Quiñones (2018) proposed a novel RTA for verifying the schedulability of a single DAG task supporting heterogeneous computing. The authors considered a parallel heterogeneous architecture composed of a host processor with m identical processors and a single accelerator device (e.g. a FPGA, GPU, etc.). Moreover, they considered a host-centric acceleration model in which the host offloads the data to the accelerator device and collects results.

The task model is the same as the DAG task model, but in the set of vertices V , besides traditional vertices that execute on the host, there is also a single node v_{off} that represents the workload executed in the accelerator device, named offloaded node, with its WCET C_{off} .

The proposed algorithm (i) transforms G in $G' = (V', E')$ which includes a new synchronization node v_{sync} , (ii) identifies the sub-DAG that may potentially execute in parallel with v_{off} , named $G^{Par} = (V^{Par}, E^{Par})$. The addition of v_{sync} guarantees that G^{Par} and v_{off} actually execute in parallel. However, this strategy may impact the average performance of the tasks due to: (i) the longest path can potentially enlarge, and (ii) the potential parallelism is reduced because of the synchronization point. An example of the model is depicted in Fig. 5.

For the schedulability test, they first computed the WCRT on the homogeneous cores $R^{hom}(\tau)$ using the Graham bound Eq. 7 (Theorem 3.9). Then they derived a new RTA supporting heterogeneous computing:

Theorem 6.1 (Serrano and Quiñones 2018) *Consider an heterogeneous DAG task τ' . Depending on the execution scenario, its response time upper bound is computed as follows:*

1. v_{Off} does not belong to the longest path.

$$R^{het}(\tau') = len(G') + \frac{1}{m} (vol(G') - len(G') - C_{Off}) \quad (34)$$

2. v_{Off} belongs to the longest path and $C_{Off} \geq R^{hom}(G^{Par})$

$$R^{het}(\tau') = len(G') + \frac{1}{m} (vol(G') - len(G') - vol(G^{Par})) \quad (35)$$

3. v_{Off} belongs to the longest path and $C_{Off} \leq R^{hom}(G^{Par})$

$$R^{het}(\tau') = len(G') - C_{Off} + len(G^{Par}) + \frac{1}{m} (vol(G') - len(G') - len(G^{Par})) \quad (36)$$

Han et al. (2019) studied the WCRT analysis of typed scheduling of a single parallel DAG task on heterogeneous multi-cores, where the workload of each vertex in the DAG can only execute on a particular type of cores.

The authors proposed two new WCRT bounds, considering and formally describing the typed DAG task model. A typed DAG is a DAG task in which each vertex has a type, that is the type of core on which the vertex should be executed. S is the set of core types, and for each $s \in S$ there are M_s cores of this type ($M_s \geq 1$). The type function $\gamma : V \times S$ characterizes the type of each vertex, i.e., $\gamma(v) = s$, where $s \in S$, denotes vertex v must be executed on cores of type s .

Then, they introduced the concept of scaled graph: the scaled graph $\hat{G}$ of G has the same topology (V and E) and type function γ as G , but vertices have different WCET

$$\forall v_i \in V : \hat{C}_i = C_i \times \left(1 - \frac{1}{M_{\gamma(v_i)}} \right) \quad (37)$$

Therefore they derived a schedulability test with overall time complexity of $O(|V| + |E|)$.

Theorem 6.2 (Han et al. 2019) *The worst-case response time of single parallel typed DAG task $\tau = (G, T, D)$ with implicit deadlines on heterogeneous multi-cores is bounded by*

$$R(G) \leq L(\hat{G}) + \sum_{s \in S} \frac{vol_s(G)}{M_s} \quad (38)$$

where $\hat{G}$ is the scaled graph of G and $vol_s(G)$ is the volume of the DAG G computed on the core type s .

As a second bound, the authors proposed a method that explores task graph structure information to improve the accuracy, however, this method is more expensive at computational level. In this sense, the authors proved that computing the second bound is a strong NP-hard problem if the number of types is a variable. In this sense, authors devise an algorithm that has polynomial time complexity if the number of types is fixed.

Chang et al. (2020) proposed a novel scheduling algorithm for the single typed DAG task on heterogeneous multi-cores, based on the criticality allocation strategy.

The strategy used for designating task-criticality, assigns each vertex v_j a differing criticality $Crit_j$ that relies upon the remaining workload of the vertex. The vertex with higher criticality is more important and consequently more urgent to be executed. Specifically, the criticality determines the order in which vertices are scheduled. Through this scheduling algorithm, the set of vertices that will block the vertices on each path can be more precisely decided. In this line, a novel scheduling algorithm is introduced to decrease the range of possible parallel execution vertices with the same type and different paths blocking each other. Then, using this scheduling algorithm as a basis, a new RTA method is presented to remove unnecessary blocking time.

Theorem 6.3 (Chang et al. 2020) *The worst-case response time of single parallel typed DAG task $\tau = (G, T, D)$ with constrained deadlines on heterogeneous multi-cores is bounded by*

$$R(G) \leq L(G) + \sum_{k=1}^{nt} \max_{s \in \Delta(Crit_k)} \inf(Crit_k, s) \quad (39)$$

where s is the core type, $nt = L(G) - C_{src} - C_{sink}$, $\Delta(Crit_k)$ is the set of types of nodes in criticality set $Crit_k$ and $\inf(Crit_k, s)$ represents the time that nodes of type s block each other in $Crit_k$.

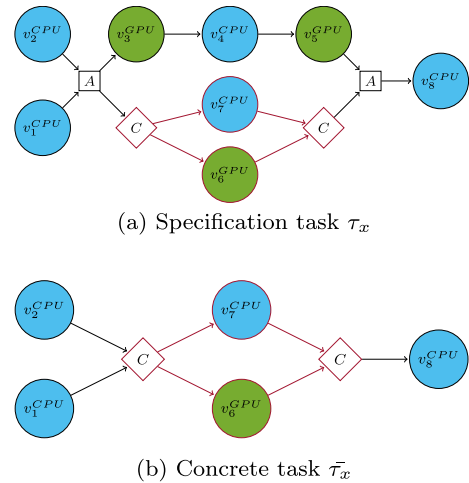
The results demonstrate that the response time upper bound is considerably better than the bound proposed by Serrano and Quiñones (2018) and Han et al. (2019) that are limited and pessimistic.

6.3 Heterogeneous conditional DAG task

Finally, the most expressive extension of the DAG task model is the one that combines the two previous ones: the Heterogeneous Conditional DAG (HC-DAG) task model. Even though this is the model that best describes the complexity of modern real applications, the literature in this context is very scarce and requires more attention.

Zahaf et al. (2019, 2020) proposed a novel real-time application model, called Heterogeneous Parallel Condition Directed Acyclic Graph Model (HPC-DAG), specifically devised for heterogeneous platforms.

Fig. 6 Example of HPC-DAG task. The alternative nodes in the specification task in **a** are depicted with a rectangle, while tags (in this case CPU and GPU) are represented with filling colors (resp. light blue and green). The task shown in **b** is one of the possible concrete tasks for the given specification task (Color figure online)



An HPC-DAG provides a rich semantic that allows the system designer to define different implementations of a software component for particular computing engines, as well as conditional branches to model if-then-else statements.

The authors modeled a heterogeneous architecture as a collection of execution engines, characterized by (i) its execution capabilities, and (ii) its scheduling policy (e.g. FTP or EDF), which can be non-preemptive or preemptive. Each computing engine has its own scheduler and a separate ready-queue. Given the importance of communication of real-time tasks on heterogeneous architectures, copy engines are treated as processing units, in which schedule communication tasks are scheduled.

Even though the model is generic, this work focus on sporadic task model with constrained deadlines under P-FP-EDF scheduling.

In the proposed model two kinds of tasks need to be clarified: (i) the specification task τ_x and (ii) the concrete task $\bar{\tau}_x$. The specification task τ_x is an extension of the C-DAG (introduced in Sect. 6.1). In addition, each vertex v_i of the task τ_x has a tag that represents the engines where it is eligible to execute onto. Besides regular and conditional nodes, in the specification task there exist also alternative nodes, which represent alternative implementations of parts of the graph/task. A concrete task $\bar{\tau}_x$ is an instance of a specification task where the different alternatives have not been considered by making implementation choices. Informally, we could say that the specification task represents the HPC-DAG model, while the concrete task is a C-DAG with tags for specific engines. An example is given in Fig. 6.

Before computing the partitioning and scheduling of the sub-tasks on the processing engines, one of the possible concrete tasks needs to be chosen. Since the number of different combinations can be very large, the authors proposed a heuristic algorithm based on a greedy strategy. Additionally, to reduce the complexity of treating with precedence constraints directly, they imposed intermediate offsets and deadlines on each sub-task.

The proposed algorithm attempts to assign one single task specification at a time: it generates all concrete tasks and, for each one of them, it allocates the intermediate deadlines and offsets accounting for the slack distributions. The algorithm gives priority to single-engine allocations to reduce preemption costs. Moreover, it can be customized with four parameters:

- Sorting order of the concrete task sets: (i) sorting according to concrete tasks total execution time (ii) sorting according to less allocation on scarce resources.
- Distribution of the slack when assigning intermediate deadlines and offsets: (i) fair distribution, that designates slack as the ratio of the original slack by the number of sub-tasks in the path, (ii) proportional distribution: assigns slack according to the contribution of the sub-task WCET in the path.
- Allocation strategy: (i) best-fit, (ii) worst-fit.
- Elimination strategy of sub-tasks (when needed): (i) random selection, (ii) parallel selection, selecting a subtask that is parallel to the critical path.

The authors showed that the HPC-DAG dominates the corresponding results for the C-DAG (Melani et al. 2015).

7 Methods evaluation

This section reports all the conducted experiments to compare the introduced schedulability tests. To perform this analysis, several methods have been implemented in C++. The code is available at <https://github.com/mive93/DAG-scheduling>, so that other researchers can use it as a library, improve it, extend it or simply reproduce the experiments. Moreover, we provide a parser to transform AMALTHEA models into DOT (graph description language). AMALTHEA is a well-known modeling language widely used in the automotive domain. This effort contributes to ease the integration of the hereafter presented methods in an available open tool platform.

7.1 AMALTHEA parser

APP4MC⁵ is an open-source framework that provides an XML-based document format for modeling embedded multi-/many-core systems. It is maintained by a consortium of academic and industrial entities from the automotive and embedded domains. The APP4MC platform provides an entire toolchain for tracing, partitioning, mapping, and modeling (Wurst et al. 2019), Rehm et al. (2021). It is based on the Eclipse modeling framework (EMF) (Steinberg et al. 2009) to model software, hardware, timing behavior and constraints of the system, supporting the AUTOSAR (AUT 2014) standard. Thanks to its simple and elegant XML-based layout, this

⁵ Application Platform Project for MultiCore (APP4MC), <http://www.eclipse.org/app4mc/>.

modeling framework is the perfect solution for the Verification and Validation (V & V) of automotive software and hardware components.

Specifically, APP4MC provides a meta-model called AMALTHEA that allows modeling both high-level requirements and low-level behavior of software components, e.g., task periods, deadlines, memory access patterns, etc. It also allows modeling the hardware platform where the application is targeted. Such an approach permits abstracting out functional implementation details. The timing behavior of concurrent tasks can be reproduced without exposing IP-critical internal details, limiting the risks to reverse the software engineering process.

AMALTHEA task τ_x is described using a Directed Acyclic Graph (DAG) $G = (V, E)$, where V corresponds to serial sequences of nodes and E the edges that denote the precedence relationship between nodes. Each task τ_x is characterized by a period T_x , a priority P_x and a scheduling policy specified by PT_x . Each node $\tau_{x,i}$ represents a different runnable, with an associated worst-case computation time $C_{x,i}$ and a worst-case memory access size $M_{x,i}$. At runtime, a task τ_x releases a sequence of runnable. Each task needs to finish their execution before the deadline.

We developed and integrated in our tool an extension that transforms AMALTHEA models into DOT graphs. To do so, AMALTHEA provides a model property that allows defining execution orders of runnables, i.e., dependencies between runnables, called *RunnableSequencingConstraint*. The defined sequencing constraints can be used to transform the AMALTHEA model into DOT graphs expressing the same task dependencies. Please refer to the documentation and the repository for further details.⁶

7.2 Task set generation

Besides accepting DAGs in DOT format, the developed C++ library offers a procedure to randomly generate DAGs. In the literature different ways to randomly generate DAG task set for multiprocessors have been adopted:

- The method proposed by Erdős and Rényi (1959), which generates graphs.
- Uunifast-Discard (Davis and Burns 2011), that efficiently generates task utilization values for task sets with a chosen number of tasks and total utilization.
- The method proposed by Melani et al. (2015) to generate DAG and C-DAG.
- YARTISS, an open-source simulation tool written in Java (Chandarli et al. 2012).

For our implementation, we selected the method proposed by Melani et al. (2015), because it allows one to create not only DAG but also conditional DAGs, and it has been extended to generate Heterogeneous DAG also.

⁶ <https://www.eclipse.org/app4mc/help/app4mc-1.0.0/index.html>.

7.3 Selected methods

The methods investigated in the analysis are the following:

- G-FP:
 - **Graham1969** Graham (1969)
 - **Baruah2012** Baruah et al. (2012)
 - **Bonifaci2013** Bonifaci et al. (2013)
 - **Li2013** Li et al. (2013)
 - **Qamhieh2013** Qamhieh et al. (2013)
 - **Melani2015** Melani et al. (2015)
 - **Pathan2017** Pathan et al. (2017)
 - **Fonseca2017** Fonseca et al. (2017)
 - **He2019** He et al. (2019)
 - **Fonseca2019** Fonseca et al. (2019)
 - **Han2019** Han et al. (2019)
- G-LP:
 - **Serrano2016** Serrano et al. (2016)
 - **Nasri2019** Nasri et al. (2019)
- P-FP:
 - **Fonseca2016** Fonseca et al. (2016)
- P-LP:
 - **Casini2018** Casini et al. (2018a)
 - **Baruah2020** Baruah (2020)

All the methods have been implemented from zero except from Melani2015, Nasri2019 and Baruah2020, whose code was available and has been integrated into the developed library.

7.4 Comparison

In the following, two different kinds of test have been reported: (i) schedulability test varying the total utilization of a task set with multiple tasks, and (ii) schedulability test varying the total utilization of a single task. For the generation of the tasks, the parameters have been set in the same way as the original implementation by Melani et al. (2015). Moreover, the computational time of every single test has been measured in milliseconds.

The experiments presented hereafter have been conducted on an Intel i9-9900KF CPU @ 3.60GHz.

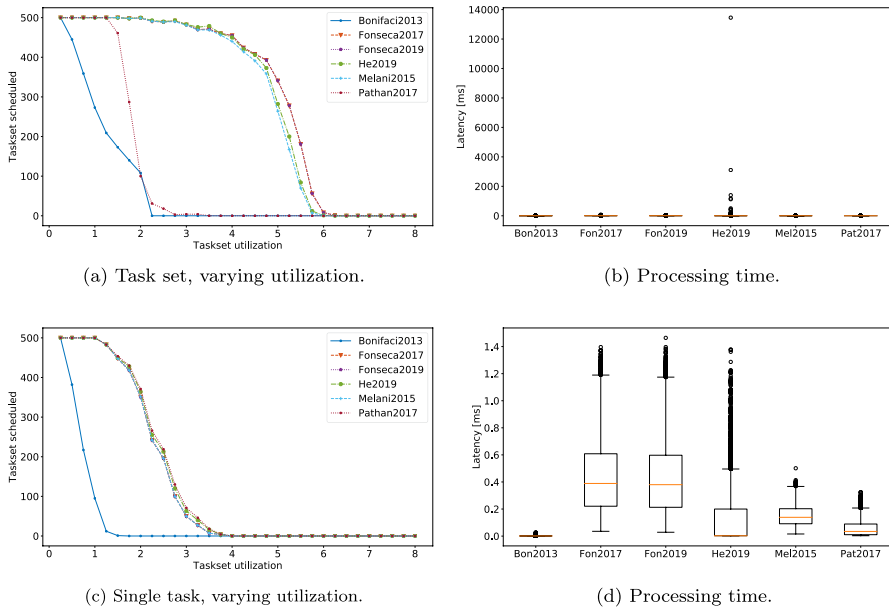


Fig. 7 Methods comparison for DAGs with FTP scheduling and implicit deadlines under G-FP scheduling

7.5 Varying U for a task set (task)

In this test, the utilization of the task set (task) is varied from $U = 0$ to $U = 8$, with an increasing step of 0.25. For each step, 500 different task set (task) are generated and tested, for a total of 16k tests. For all the experiments, we fixed the number of processors to $m = 8$.

7.6 G-FP for DAG, FTP and implicit deadlines

Figure 7 details the results for the comparison among the G-FP-FTP scheduling for DAGs with implicit deadlines. From these charts, we can notice that the best methods in terms of schedulability are Fonseca2017 and Fonseca2019, which dominate all the other approaches when considering multiple tasks. The algorithms are very similar, and we can notice that the improvements introduced in the version of 2019 are almost negligible. The execution time of these algorithms is also comparable and it takes up to ~ 3 ms. These methods are also the most complex one among all the considered. To compute the carry-out workload, they require two conversions of the DAG: first, the DAG needs to be converted into an NFJ DAG and then this needs to be decomposed in an SPTree.

When considering a single DAG, the performance of Fonseca2019, Fonseca2017, and Melani2015 are the same, because the methods differ only in the computation

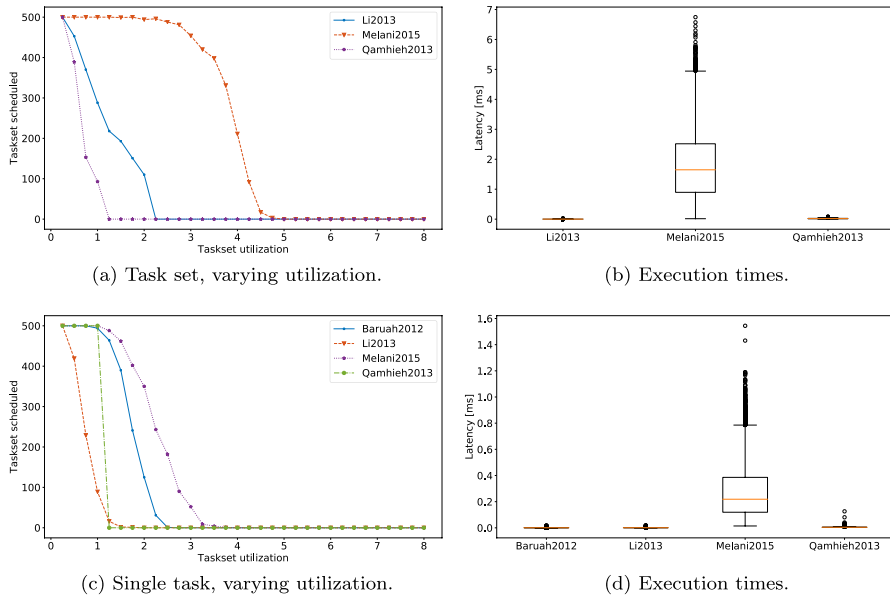


Fig. 8 Methods comparison for DAGs with EDF scheduling and implicit deadlines under G-FP scheduling

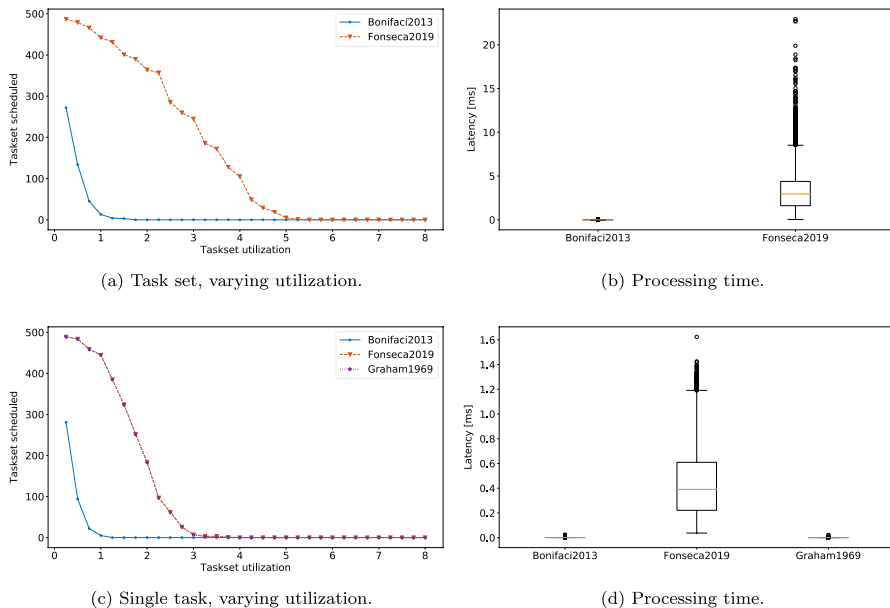


Fig. 9 Methods comparison for DAGs with FTP scheduling and arbitrary deadlines under G-FP scheduling

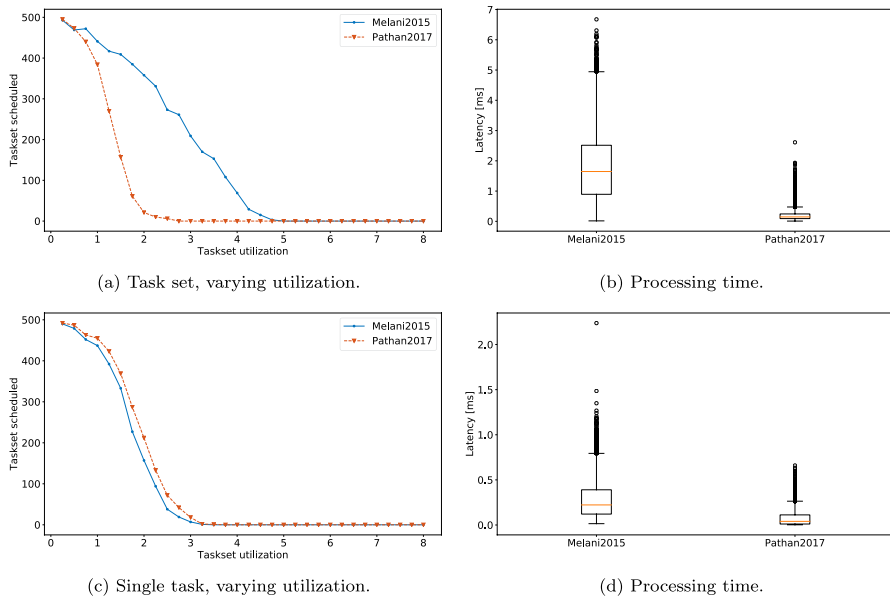


Fig. 10 Methods comparison for Conditional DAGs with FTP scheduling and constrained deadlines under G-FP scheduling

of high-priority tasks interference, which is obviously missing in the single task experiments.

He2019 has similar results with respect to Melani 2015, even though it dominates it in every test, both for multiple and single tasks. This method is the one with more variability in the execution times, reaching even more than 13 seconds for some runs (as can be seen in Fig. 7b).

Pathan2017 was supposed to always dominate Melani 2015. For the single task test, this is true, and the method slightly dominates He2019. However, when considering multiple tasks, Pathan2017 performance is poor and is greatly dominated by both He2019 and Melani2015. The problem can be found in Eq. 15: in this test the authors account for the whole workload of a task τ_y for every subtask of τ_x which is very pessimistic. Melani 2015 also accounted for the whole workload of a task τ_y in Eq. 9, but only once for task.

7.7 G-FP for DAG, EDF and implicit deadlines

Figure 8 reports the results for the comparison among the G-FP-EDF scheduling for DAGs with implicit deadlines. It is here reported the implicit case and not the constrained because the former is more comprehensive, including also Li2013 which is thought for implicit DAGs only. From these charts, we can notice that Melani2015 dominates all the other methods, both for multiple and single DAGs. This method is also the one with the highest computational time, reaching in some cases ~ 14 ms.

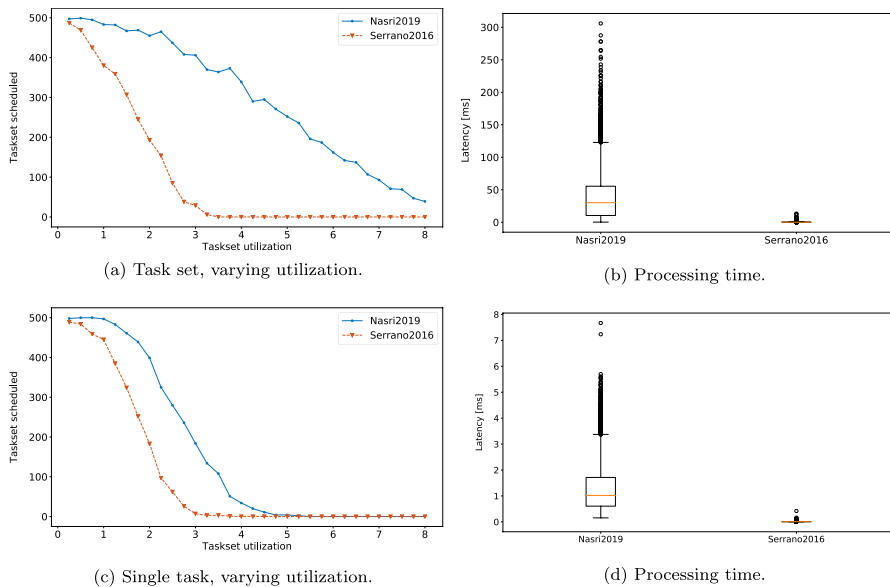


Fig. 11 Methods comparison for DAGs with FTP scheduling and constrained deadlines under G-LP

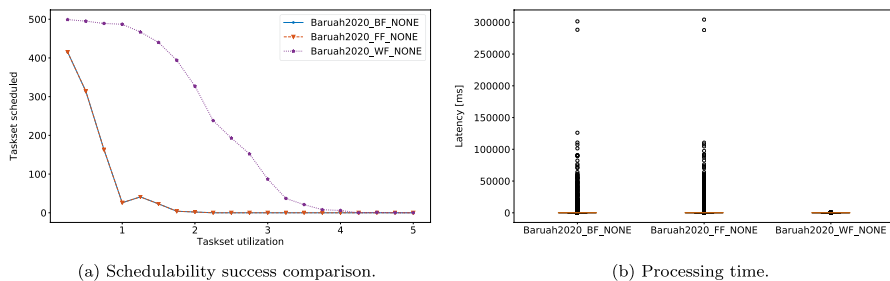


Fig. 12 Comparison of Best Fit (BF), First Fit (FF) and Worst Fit (WF) for single task with Baruah 2020 exact test. None, in the labels, indicates that no ordering of the nodes is performed

7.8 G-FP for DAG, FTP and arbitrary deadlines

Figure 9 reports the results for the comparison among the G-FP-FTP scheduling for DAGs with arbitrary deadlines. Fonseca2019 dominates Bonifaci2013 when considering multiple tasks; while it has the same performance as Graham1969 for a single task. The method in some cases reaches a computational time of ~ 20 ms.

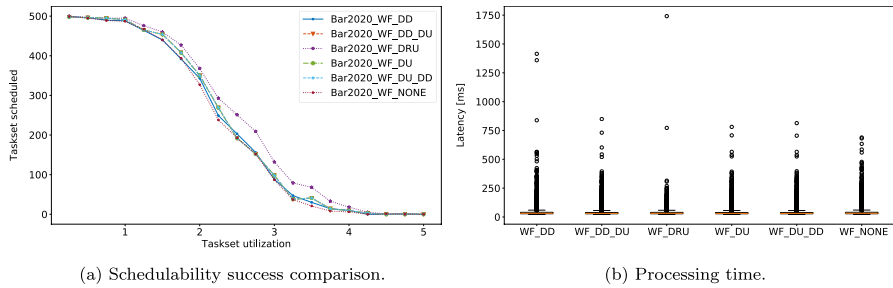


Fig. 13 Comparison of nodes ordering, under WF for single task with Baruah 2020 exact test

7.9 G-FP for C-DAG, FTP and constrained deadlines

Figure 10 reports the results for the comparison among the G-FP-FTP scheduling for conditional DAGs with constrained deadlines. Similar results with respect to the DAG case can be seen in the plots. Again, Pathan 2017 dominates Melani2015 only when a single task is considered, but it is definitely dominated in the multiple tasks tests.

7.10 G-LP for DAG, FTP and constrained deadlines

In the context of global scheduling, two methods have been developed for limited preemption, namely Serrano2016 and Nasri2019. The result of their comparison can be found in Fig. 11. In these charts, it is clear that Nasri 2019 dominates Serrano2016, having significantly better results in each performed test. However, given the greatest complexity of the Nasri2019 method, its computational time is also the highest, reaching even 300 ms for a single test.

7.11 P for DAG, FTP and implicit deadlines

Regarding partitioned scheduling, few works can be found in the literature. Indeed, the most relevant ones are Fonseca 2016, for fully preemptive policy, Casini2018 and Baruah 2020 for the limited preemptive policy. Baruah2020 can be only applied to single a DAG, but it is an exact method, based on ILP.

Partitioned methods highly depend on the partitioning that is done beforehand. Given that the node-to-core partitioning problem is NP-hard (it can be reconducted to the bin-packing problem), several heuristics have been investigated to obtain the best results.

Figure 12 shows the performance of Baruah2020 under the Worst-Fit (WF), First-Fit (FF) and Best-Fit (BF) strategies, that affect the way core are ordered. It can be

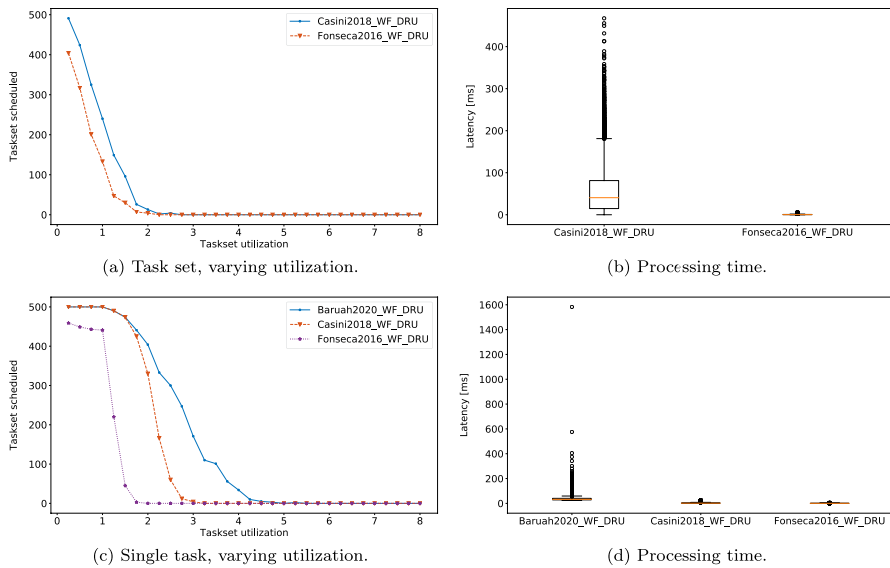


Fig. 14 Methods comparison for DAGs with FTP scheduling and implicit deadlines under partitioned scheduling

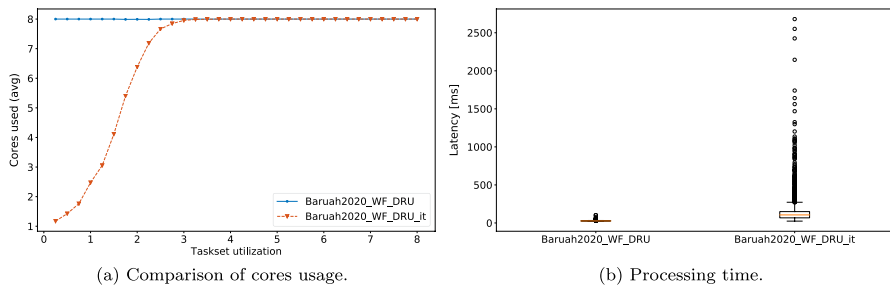


Fig. 15 Comparison of the regular version of Baruah 2020 for single DAG and its iterative version to find the minimum number of processors needed to schedule the DAG

easily noticed that the WF strategy strongly dominates the other two, not only in terms of schedulability but also of execution time.

Figure 13 reports instead the schedulability results varying the strategy used to order the nodes, when applying WF. Six different approaches are compared: (i) NONE does not order the nodes of the tasks in any manner; (ii) Decreasing Utilization (DU) order the nodes for their utilization; (iii) Decreasing Density (DD) orders them for their density (iv) Decreasing Utilization Decreasing Density (DUDD) orders the nodes by decreasing utilization first and decreasing density second (v) Decreasing Density Decreasing Utilization (DDDU) does the opposite; (vi) Decreasing Remaining Utilization (DRU) orders the nodes by the total utilization of their successors. *WF_NONE* is the strategy leading to the poorest results, while

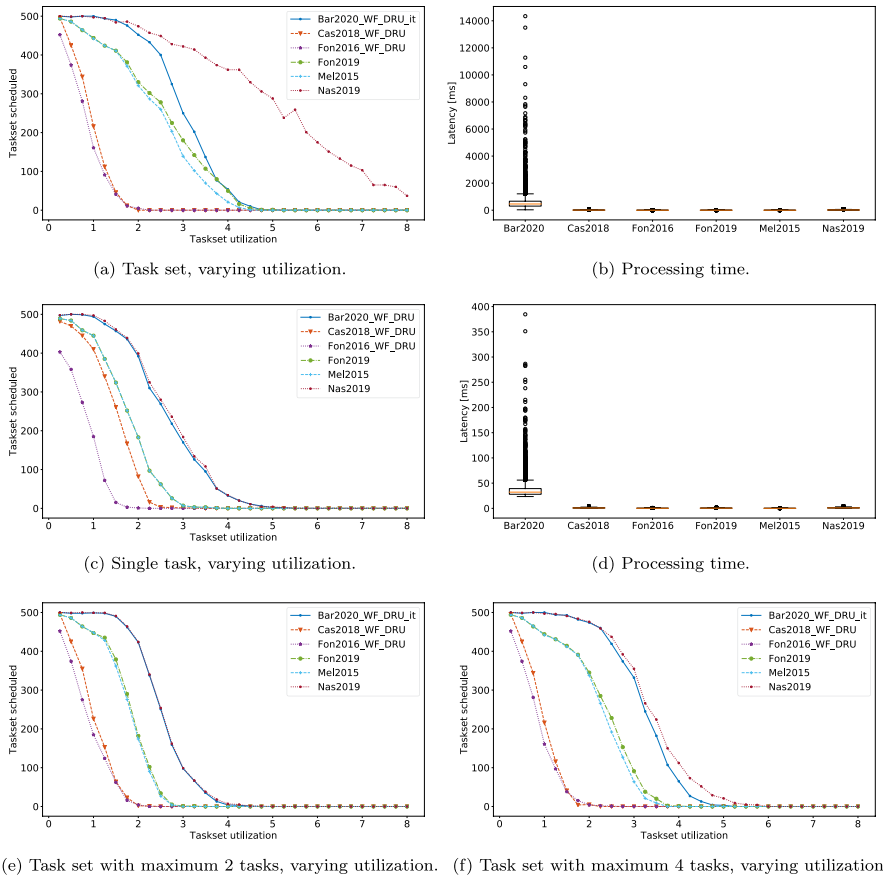


Fig. 16 SOTA methods comparison for DAGs with FTP scheduling and constrained deadlines

WF_DRU is the best one. The rationale is to split on different cores the nodes that unlock the execution of the rest of the DAGs, so that those run in parallel. Given the results of the analysis, *WF_DRU* has been chosen to allocate the nodes for every partitioned method.

Finally, Fig. 14 shows the comparison of these partitioned methods, where it can be noticed that Casini2018 always dominates Fonseca2016, while being slower than it, reaching even ~ 500 ms for a single run. Baruah clearly dominates the other two methods, but it is an order of magnitude slower w.r.t. the sufficient tests, reaching almost 2 s for a single test.

7.12 State-Of-The-Art methods for DAG, FTP and constrained deadlines

Finally, the best methods in terms of schedulability have been considered altogether. To compare Baruah2020 with the other algorithms for multiple tasks, an iterative version has been implemented, to obtain the minimum number of cores

needed for a single DAG. Figure 15 shows the comparison among the default version of Baruah2020 and its iterative variant for a single DAG, showing the average cores used and the execution times of the two methods. Clearly, the iterative version requires using fewer cores, especially when the utilization is low. However, the latency of the method increases by m times. Using this iterative version, a task set can be scheduled via Baruah2020, partitioning tasks ordered by decreasing utilization on the available cores.

Figure 16 reports the comparison among Baruah2020 (P-LP), Casini2018 (P-LP), Fonseca2016 (P-FP), Fonseca2019 (G-FP), Melani2015 (G-FP), Nasri2019 (G-LP).

Nasri2019 dominates each other methods when considering a task set of DAGs, while the poorest performance is given by Fonseca2016, which is dominated by all the others. When considering a single DAG, Nasri2019 and Baruah2020 have very similar performances and dominate the remaining approaches. Again, for single task test, Fonseca2019 and Melani2015 have the same results.

It is though important to remember that comparing Baruah2020 and Nasri2019 is not fair: Baruah2020 is exact for a single DAG, and it has been adapted to run on a task set partitioning tasks to cores. This technique, however, immediately fails when the number of tasks is greater than the number of available cores. Moreover, it could happen to allocate a whole core for a negligible node, which is never possible for a global method. Therefore, to be fairer w.r.t. iterative Baruah2020, the schedulability ratio results for task sets with maximum 2 or 4 tasks are reported in Fig. 16e and f. From these charts, it can be noticed that the performance of Baruah2020 and Nasri2019 in terms of schedulability are comparable.

Regarding the execution time, Baruah2020 is the one with the highest computational time, reaching even 14 s in its iterative version.

8 Discussion

8.1 Global scheduling

When considering global scheduling for a DAG tasks or a task set, the three SOTA methods are Nasri2019, Fonseca2019, and Melani2015 (Fig. 16a). Nasri2019 outperforms every method analyzed, and it is for sure the best method for G-LP (Fig. 16a). However, it is also the method with the highest computational time (Fig. 11b), due to the expensive exploration in the possibilities space. Indeed, it can not be used for online purposes, to handle a dynamic task set, but only for offline static ones. Moreover, the actual performance could be even worse: for the performed tests the number of nodes in the random generated DAGs was limited, varying from 4 to 37 nodes; in the original paper (Nasri et al. 2019) the authors state that execution times for tests become intractable for DAGs with more than 64 nodes, making the method not suitable for those cases. Fonseca2019 is the method to adopt when using the DAG task model with G-FP-FTP scheduling, being the one that dominates all the other G-FP solutions both for constrained and arbitrary deadlines

(Fig. 9), especially when considering multiple tasks. Melani2015 is the best solution for multiple and single tasks for the DAG task model when using EDF. When considering G-FP for C-DAGs, it is still the best solution for EDF (single and multiple tasks) and for multiple tasks when using FTP. However, when using G-FP-FTP for a single task, Melani is dominated by Pathan2017 (Fig. 10c).

8.2 Partitioned scheduling

When considering partitioned scheduling, the allocation strategy should be chosen carefully because it affects the scheduling results. It has been shown that the best heuristic to partition nodes to cores is using the Worst-Fit strategy, having ordered the nodes by their Decreasing Remaining Utilization value. The three SOTA methods are Fonseca2016 (P-FP), Casini2018 (P-LP) and Baruah2020 (P-LP). The latter is an exact test and dominates the other two (Fig. 14c), but is only thought for single DAG and it has a high computational time. Among the two remaining, Casini2018 is the best (Fig. 14a). Nonetheless, we have still to remember that those methods, even Casini2018 and Fonseca2016, become too expensive in terms of execution time when the DAG has several nodes: indeed their analyses are based on the complete exploration of all the possible paths in a graph, which is known to be of exponential complexity.

8.3 Global vs partitioned

In conclusion, what is the best strategy for scheduling DAGs, partitioned or global scheduling? Regarding a DAG single task, the performance of the best global method, Nasri2019, and the exact partitioned method, Baruah 2020, are comparable (Fig. 16c). In the task set case, the results show that Nasri2019 is the best method available, but Baruah 2020, used in its iterative form, can not be considered exact (Fig. 16e, f). In addition, partitioned methods would require an exact allocation strategy for a fair comparison, which is not the case for these experiments. Therefore, a clear answer is missing. Schedulability tests are not sufficient to answer this delicate and important question: one should also compare actual response times, end-to-end latencies and account for migration overheads. Moreover, additional tests varying all the DAG generator parameters should be performed to have a clearer overview.

However, we can notice that the method with the best trade-off among computational time, schedulability, and easy-adoption is Melani2015. It has medium schedulability ratio performance, however, it has several advantages: (i) it has a very small computational time (~ 4 ms in the worst case among all the performed tests) which make it possible to use it as an online method; (ii) it can be applied both for EDF, FTP and any work-conserving scheduling algorithms; (iii) it can be easily extended for G-LP (i.e. Serrano2016); (iv) it is also valid for C-DAGs; (v) it can handle DAGs with hundreds of nodes in feasible time (e.g. the worst-case computational time for task sets having among 5 to 30 tasks with 50 to 300 nodes is ~ 700 ms); (vi) it is easy to implement.

9 Conclusion

This paper reported an organized survey of the real-time DAG literature, analyzing different DAG models (i.e. DAG, C-DAG, H-DAG, HC-DAG), for global, partitioned and federated scheduling, considering different scheduling algorithms (e.g. EDF, FTP), preemption policies (i.e. FP, LP, FNP) and deadlines types (i.e. constrained, implicit and arbitrary). Besides the survey, a library comprehensive of all the SOTA methods has been implemented, which made a fair evaluation of the different methods possible.

Two goals have been accomplished. First, it emerged that the most suitable model for new-generation real-time system is the Heterogeneous Conditional-DAG, which comprises the concepts of parallelism, conditionality, and heterogeneity in a single model. Second, a collection point for a clear and fair comparison among methods related to the DAG task model and its extensions has been achieved, thanks to the joint efforts of translating each method into a common convention and implementing them in a single library.

The hope is that the real-time community will use this work as a point of reference, to start working on this subject and improve the state-of-the-art.

References

- (2014) Autosar—specification of timing extensions. Tech. rep
- Baruah SK (2006) The non-preemptive scheduling of periodic tasks upon multiprocessors. *Real-Time Syst* 32(1–2):9–20
- Baruah S (2007) Techniques for multiprocessor global schedulability analysis. In: 28th IEEE international real-time systems symposium (RTSS 2007), IEEE, pp 119–128
- Baruah S (2014) Improved multiprocessor global schedulability analysis of sporadic dag task systems. In: Real-time systems (ECRTS), 2014 26th Euromicro conference on, IEEE, pp 97–105
- Baruah S (2015a) The federated scheduling of constrained-deadline sporadic dag task systems. In: 2015 design, automation & test in Europe conference & exhibition (DATE), IEEE, pp 1323–1328
- Baruah S (2015b) Federated scheduling of sporadic dag task systems. In: 2015 IEEE international parallel and distributed processing symposium, IEEE, pp 179–186
- Baruah S (2020) Scheduling dags when processor assignments are specified. In: Proceedings of the 28th international conference on real-time networks and systems, pp 111–116
- Baruah S, Burns A (2006) Sustainable scheduling analysis. In: 2006 27th IEEE international real-time systems symposium (RTSS'06), IEEE, pp 159–168
- Baruah SK, Mok AK, Rosier LE (1990) Preemptively scheduling hard-real-time sporadic tasks on one processor. In: [1990] Proceedings 11th real-time systems symposium, IEEE, pp 182–190
- Baruah SK, Chen D, Gorinsky S, Mok AK (1999) Generalized multiframe tasks. *Real-Time Syst* 17:5–22
- Baruah S, Bonifaci V, Marchetti-Spaccamela A, Stougie L, Wiese A (2012) A generalized parallel task model for recurrent real-time processes. In: 2012 IEEE 33rd real-time systems symposium (RTSS 2012), IEEE, pp 63–72
- Baruah S, Bertogna M, Buttazzo G (2015a) Multiprocessor scheduling for real-time systems
- Baruah S, Bonifaci V, Marchetti-Spaccamela A (2015b) The global EDF scheduling of systems of conditional sporadic dag tasks. In: Real-time systems (ECRTS), 2015 27th Euromicro conference on, IEEE, pp 222–231
- Bertogna M, Baruah S (2010) Limited preemption edf scheduling of sporadic task systems. *IEEE Trans Industr Inf* 6(4):579–591
- Bletsas K (2007) Worst-case and best-case timing analysis for real-time embedded systems with limited parallelism. Citeseer

- Bonifaci V, Marchetti-Spaccamela A, Stiller S, Wiese A (2013) Feasibility analysis in the sporadic dag task model. In: Real-time systems (ECRTS), 2013 25th Euromicro conference on, IEEE, pp 225–233
- Burgio P, Cavicchioli R, Verucchi M (2019) Real-time heterogeneous platforms. In: Heterogeneous computing architectures, CRC Press, pp 233–259
- Burmyakov A, Bini E, Tovar E (2015) An exact schedulability test for global FP using state space pruning. In: Proceedings of the 23rd international conference on real time and networks systems, pp 225–234
- Burns A, Baruah SK et al (2008) Sustainability in real-time scheduling. *JCSE* 2(1):74–97
- Buttazzo G (2004) Hard real-time computing systems: predictable scheduling algorithms and applications. Real-time systems series, Springer. <https://books.google.it/books?id=fpJAZM6FK2sC>
- Buttazzo GC, Lipari G, Abeni L (1998) Elastic task model for adaptive rate control. In: RTSS
- Buttazzo GC, Bertogna M, Yao G (2013) Limited preemptive scheduling for real-time systems. a survey. *IEEE Trans Industr Inf* 9:3–15
- Casini D, Biondi A, Nelissen G, Buttazzo G (2018a) Memory feasibility analysis of parallel tasks running on scratchpad-based architectures. In: 2018 IEEE real-time systems symposium (RTSS), IEEE, pp 312–324
- Casini D, Biondi A, Nelissen G, Buttazzo G (2018b) Partitioned fixed-priority scheduling of parallel tasks without preemptions. In: 2018 IEEE real-time systems symposium (RTSS), IEEE, pp 421–433
- Casini D, Biondi A, Nelissen G, Buttazzo G (2020) A holistic memory contention analysis for parallel real-time tasks under partitioned scheduling. In: 2020 IEEE real-time and embedded technology and applications symposium (RTAS), IEEE, pp 239–252
- Chandarli Y, Fauberteau F, Masson D, Midonnet S, Qamhieh M (2012) Yartiss: a tool to visualize, test, compare and evaluate real-time scheduling algorithms
- Chang S, Zhao X, Liu Z, Deng Q (2020) Real-time scheduling and analysis of parallel tasks on heterogeneous multi-cores. *J Syst Architect* 105:101704
- Chen JJ, Nelissen G, Huang WH, Yang M, Brandenburg B, Bletsas K, Liu C, Richard P, Ridouard F, Audsley N et al (2019) Many suspensions, many problems: a review of self-suspending tasks in real-time systems. *Real-Time Syst* 55(1):144–207
- Cormen TH, Leiserson CE, Rivest RL, Stein C (2009) Introduction to algorithms. MIT Press, Cambridge
- Davis RI, Burns A (2011) Improved priority assignment for global fixed priority pre-emptive scheduling in multiprocessor real-time systems. *Real-Time Systems* 47(1):1–40
- Dong Z, Liu C (2019) An efficient utilization-based test for scheduling hard real-time sporadic dag task systems on multiprocessors. In: 2019 IEEE real-time systems symposium (RTSS), IEEE, pp 181–193
- Erdős P, Rényi A (1959) On random graphs i. *Publicationes Mathematicae Debrecen* 6:290–297
- Fonseca JC, Nélis V, Raravi G, Pinho LM (2015) A multi-dag model for real-time parallel applications with conditional execution. In: Proceedings of the 30th annual ACM symposium on applied computing, ACM, pp 1925–1932
- Fonseca J, Nelissen G, Nélis V, Pinho LM (2016) Response time analysis of sporadic DAG tasks under partitioned scheduling. In: 2016 11th IEEE symposium on industrial embedded systems (SIES), IEEE, pp 1–10
- Fonseca J, Nelissen G, Nélis V (2017) Improved response time analysis of sporadic dag tasks for global fp scheduling. In: Proceedings of the 25th international conference on real-time networks and systems, pp 28–37
- Fonseca J, Nelissen G, Nélis V (2019) Schedulability analysis of dag tasks with arbitrary deadlines under global fixed-priority scheduling. *Real-Time Syst* 55(2):387–432
- Graham RL (1969) Bounds on multiprocessor timing anomalies. *SIAM J Appl Math* 17:263–269
- Guan F, Qiao J, Han Y (2020) Dag-fluid: a real-time scheduling algorithm for DAGs. *IEEE Trans Comput*
- Guo Z, Bhuiyan A, Saifullah A, Guan N, Xiong H (2017) Energy-efficient multi-core scheduling for real-time dag tasks. In: 29th Euromicro conference on real-time systems (ECRTS 2017), Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik
- Han M, Guan N, Sun J, He Q, Deng Q, Liu W (2019) Response time bounds for typed dag parallel tasks on heterogeneous multi-cores. *IEEE Trans Parallel Distrib Syst* 30(11):2567–2581
- He Q, Guan N, Guo Z et al (2019) Intra-task priority assignment in real-time scheduling of dag tasks on multi-cores. *IEEE Trans Parallel Distrib Syst* 30(10):2283–2295
- Jiang X, Long X, Guan N, Wan H (2016) On the decomposition-based global EDF scheduling of parallel real-time tasks. In: 2016 IEEE real-time systems symposium (RTSS), pp 237–246, <https://doi.org/10.1109/RTSS.2016.031>

- Jiang X, Guan N, Liu D, Liu W (2019) Analyzing GEDF scheduling for parallel real-time tasks with arbitrary deadlines. In: 2019 design, automation & test in Europe conference & exhibition (DATE), IEEE, pp 1537–1542
- Li J, Agrawal K, Lu C, Gill C (2013) Outstanding paper award: analysis of global EDF for parallel tasks. In: real-time systems (ECRTS), 2013 25th Euromicro conference on, IEEE, pp 3–13
- Li J, Chen JJ, Agrawal K, Lu C, Gill C, Saifullah A (2014) Analysis of federated and global scheduling for parallel real-time tasks. In: Real-time systems (ECRTS), 2014 26th Euromicro conference on, IEEE, pp 85–96
- Liu C, Anderson JH (2010) Supporting soft real-time dag-based systems on multiprocessors with no utilization loss. In: 2010 31st IEEE real-time systems symposium, IEEE, pp 3–13
- Liu C, Anderson JH (2011) Supporting graph-based real-time applications in distributed systems. In: 2011 IEEE 17th international conference on embedded and real-time computing systems and applications, IEEE, vol 1, pp 143–152
- Liu CL, Layland JW (1973) Scheduling algorithms for multiprogramming in a hard-real-time environment. *J ACM* 20:46–61
- Lukkien JJ, Verhaegh WF, Bril RJ (2007) Worst-case response time analysis of real-time tasks under fixed-priority scheduling with deferred preemption revisited. In: 19th Euromicro conference on real-time systems (ECRTS'07) (ECRTS), pp 269–279, <https://doi.org/10.1109/ECRTS.2007.38>
- Melani A, Bertogna M, Bonifaci V, Marchetti Spaccamela A, Buttazzo GC (2015) Response-time analysis of conditional dag tasks in multiprocessor systems. In: Real-time systems (ECRTS), 2015 27th Euromicro conference on, IEEE, pp 211–221
- Mok AK, Chen D (1996) A multiframe model for real-time tasks. In: RTSS
- Nasri M, Nelissen G, Brandenburg BB (2019) Response-time analysis of limited-preemptive parallel dag tasks under global scheduling. In: 31st Conference on Real-Time Systems, pp 21–1
- Nelissen G, Fonseca J, Raravi V, Nélis V (2015) Timing analysis of fixed priority self-suspending sporadic tasks. In: 2015 27th Euromicro conference on real-time systems, IEEE, pp 80–89
- Parri A, Biondi A, Marinoni M (2015) Response time analysis for G-EDF and G-DM scheduling of sporadic dag-tasks with arbitrary deadline. In: Proceedings of the 23rd international conference on real time and networks systems, pp 205–214
- Pathan R, Voudouris P, Stenström P (2017) Scheduling parallel real-time recurrent tasks on multicore platforms. *IEEE Trans Parallel Distrib Syst* 29(4):915–928
- Qamhieh M, Midonnet S (2014) An experimental analysis of dag scheduling methods in hard real-time multiprocessor systems. In: Proceedings of the 2014 conference on research in adaptive and convergent systems, pp 284–290
- Qamhieh M, Midonnet S, George L (2012) Dynamic scheduling algorithm for parallel real-time graph tasks. *ACM SIGBED Revw* 9(4):25–28
- Qamhieh M, Fauberteau F, George L, Midonnet S (2013) Global EDF scheduling of directed acyclic graphs on multiprocessor systems. In: Proceedings of the 21st international conference on real-time networks and systems, pp 287–296
- Rehm F, Dasari D, Hamann A, Pressler M, Ziegenbein D, Seitter J, Sañudo I, Capodiec N, Burgio P, Bertogna M (2021) Performance modeling of heterogeneous hw platforms. *Microprocess Microsyst*
- Saifullah A, Ferry D, Li J, Agrawal K, Lu C, Gill CD (2014) Parallel real-time scheduling of DAGs. *IEEE Trans Parallel Distrib Syst* 25(12):3242–3252
- Serrano MA, Quiñones E (2018) Response-time analysis of dag tasks supporting heterogeneous computing. In: Proceedings of the 55th annual design automation conference, pp 1–6
- Serrano MA, Melani A, Bertogna M, Quinones E (2016) Response-time analysis of dag tasks under fixed priority scheduling with limited preemptions. In: Proceedings of the 2016 conference on design, automation & test in Europe, EDA Consortium, pp 1066–1071
- Serrano MA, Melani A, Kehr S, Bertogna M, Quinones E (2017) An analysis of lazy and eager limited preemption approaches under dag-based global fixed priority scheduling. In: Real-time distributed computing (ISORC), 2017 IEEE 20th international symposium on, IEEE, pp 193–202
- Steinberg D, Budinsky F, Paternostro M, Merks E (2009) EMF: eclipse modeling framework 2.0, 2nd edn. Addison-Wesley Professional
- Stigge M, Ekberg P, Guan N, Yi W (2011) The digraph real-time task model. In: 2011 17th IEEE real-time and embedded technology and applications symposium, pp 71–80
- Sun J, Guan N, Sun J, Chi Y (2019) Calculating response-time bounds for OpenMP task systems with conditional branches. In: 2019 IEEE real-time and embedded technology and applications symposium (RTAS), IEEE, pp 169–181

- Sun J, Li F, Guan N, Zhu W, Xiang M, Guo Z, Yi W (2020) On computing exact WCRT for dag tasks. In: 2020 57th ACM/IEEE design automation conference (DAC), IEEE, pp 1–6
- Thekkilakattil A, Davis RI, Dobrin R, Punnekkat S, Bertogna M (2015) Multiprocessor fixed priority scheduling with limited preemptions. In: Proceedings of the 23rd international conference on real time and networks systems, pp 13–22
- Ueter N, von der Brüggen G, Chen JJ, Li J, Agrawal K (2018) Reservation-based federated scheduling for parallel real-time tasks. In: 2018 IEEE real-time systems symposium (RTSS), pp 482–494, <https://doi.org/10.1109/RTSS.2018.00061>
- Ullman JD (1975) Np-complete scheduling problems. *J Comput Syst Sci* 10(3):384–393
- Valdes J, Tarjan RE, Lawler EL (1979) The recognition of series parallel digraphs. In: Proceedings of the eleventh annual ACM symposium on theory of computing, pp 1–12
- Vestal S (2007) Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance. In: 28th IEEE international real-time systems symposium (RTSS 2007), pp 239–243, <https://doi.org/10.1109/RTSS.2007.47>
- Wurst F, Dasari D, Hamann A, Ziegenbein D, Sañudo I, Capodiecì N, Bertogna M, Burgio P (2019) System performance modelling of heterogeneous hw platforms: an automated driving case study. In: 2019 22nd Euromicro conference on digital system design (DSD), pp 365–372, <https://doi.org/10.1109/DSD.2019.00060>
- Yalcinkaya B, Nasri M, Brandenburg BB (2019) An exact schedulability test for non-preemptive self-suspending real-time tasks. In: 2019 design, automation & test in Europe conference & exhibition (DATE), IEEE, pp 1228–1233
- Yang K, Yang M, Anderson JH (2016) Reducing response-time bounds for dag-based task systems on heterogeneous multicore platforms. In: Proceedings of the 24th international conference on real-time networks and systems, pp 349–358
- Yang T, Deng Q, Sun L (2019) Building real-time parallel task systems on multi-cores: a hierarchical scheduling approach. *J Syst Architect* 92:1–11
- Zahaf HE, Capodiecì N, Cavicchioli R, Bertogna M, Lipari G (2019) A c-dag task model for scheduling complex real-time tasks on heterogeneous platforms: preemption matters. arXiv preprint [arXiv:1901.02450](https://arxiv.org/abs/1901.02450)
- Zahaf HE, Houssam-Eddine Z, Capodiecì N, Cavicchioli R, Lipari G, Bertogna M (2020) The hpc-dag task model for heterogeneous real-time systems. *IEEE Trans Comput* 70(10):1747–61

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Micaela Verucchi is a computer scientist and engineer with expertise in high-performance computing platforms, autonomous systems, perception, and software engineering. She holds a BSc in Computer Science and an MSc in Computer Engineering from the University of Modena and Reggio Emilia, and obtained her Ph.D. in Computer Science from the same university in 2021. During her Ph.D., she focused on real-time systems and artificial intelligence. Micaela has worked as a Perception Engineer for Autonomous Racing in the Indy Autonomous Challenge since 2021, and currently serves as a Project Manager at Hipert Srl. Her research and practical experience contribute to the advancement of these fields.



Ignacio Sañudo Olmedo obtained his B.Sc. and M.Sc. in Computer Science Engineering from the University of Cantabria, Spain, in 2014, and his Ph.D. in Computer Science from the University of Modena and Reggio Emilia, Italy, in 2018. He currently holds the position of Project Manager at Hipert Srl. His research focuses on high-performance computing platforms, autonomous and intelligent systems, as well as safety, reliability, and software engineering.



Marko Bertogna is a Full Professor at the University of Modena and Reggio Emilia, where he directs the High-Performance Real-Time (HiPeRT) laboratory. He is a member of the Board of Directors of the UniverMantova Foundation. His main research interests are high-performance real-time systems, particularly for autonomous driving applications and industrial automation. He was a Researcher at the Sant'Anna School of Pisa, where he obtained a Ph.D. in Computer Science with a thesis awarded as the best doctoral thesis of 2008–2009. He is the author of over 100 scientific articles, which have received over 3000 citations. He has received ten Best Paper Awards at international conferences. He leads several EU and industrial projects, securing over 10 million euros in funding for his research group. He is the CEO and founder of the university spinoff Hipert Srl.